

Basic Model

Contents

BasicModel	4
Basic Model of Cognition	4
Documentation	4
Overview	5
Files	5
Quick Start	5
XML Configuration	5
Output	6
Architecture	6
Overview	6
Spaces	6
Reconstruction Symbols	7
Single Optimizer with Overlapping Weight Spaces	7
Training Loop	8
Three-File Architecture	8
Language System	9
Sigma and Pi Layers	9
Dimensionality Constraints	9
Pi Layer Derivation	10
Invertible Linear Layer (LDU)	10
InvertibleLinearLayer — LDU Factorisation	10
Ergodic Noise Injection (Factor-Level)	10
stable=True	11
noise_d	11
naive Flag	11
Noise Lifecycle in Ergodic Mode	11
Class Renames	11
Ergodic Exploration	12
Basic Model	12
Spaces	19
Overview	19
Base Class: Space	19
InputSpace	19
PerceptualSpace	20
ConceptualSpace	21
SymbolicSpace	21
SyntacticSpace	22
OutputSpace	23

Language	23
Overview	23
Grammar	24
CNF and Unary Rules	24
Word Encoding	24
SymbolicSpace	25
Role	25
Forward Path	25
Reverse Path	25
Key Properties	25
SyntacticLayer	25
Architecture: Recursive Hybrid with Depth Embedding	25
Why Recursive?	26
Differentiable Rule Selection	26
SyntacticSpace	26
Role	26
Forward Path	26
Reverse Path	26
Key Properties	27
Open Question: Differentiating the Rules Themselves	27
Logic.md	27
Overview	27
1. Subsymbolic Layer	27
Operators	27
2. Symbolization	28
3. Symbolic Layer (Scalars in [-1,1])	28
Negation (affirming)	28
Non (non-affirming)	28
Union	28
Intersection	28
Parthood (order relation)	28
4. Interpretation	29
5. Key Insight	29
Ergodic Exploration via Adaptive Bias-Variance Control	29
1. The Ergodic Weights	29
2. Why Standard Adam Fails on α	30
3. The Gradient Energy Sensor	30
3.1 Modified Second-Moment Estimator	30
3.2 Bias Correction	30
3.3 What $\hat{v}_t$ Measures	31
3.4 The Alpha Update Rule	31
3.5 Layer-Local Certainty (Per-Neuron Sigma)	31
4. Comparison: Adam vs. Gradient Energy Sensor	31
5. Derivation: Why Zero-Mean Implies Drop m_t	32
6. Dropout via Bias	32
7. The <code>global_temp</code> Sensitivity Knob	33
8. Initialization and Bootstrap	33
9. Layer Architecture	33
ErgodicLayer Base Class	34
InvertibleLinearLayer (SVD decomposition)	34
InvertibleRotationLayer (Householder)	34
InvertibleDiagonalLayer	34

10. Training Loop Integration	34
11. Summary of the Full Algorithm	35
Factor-Level Noise Injection	35
Previous Approach: Matrix-Level Blending	35
New Approach: Factor-Level Injection	36
stable=True and the d Backbone	36
SigmaLayer and PiLayer Integration	36
Training	36
Overview	36
Phase 1: Embedding Pretraining (<code>make train / embed.py train</code>)	36
Pipeline	36
SBOW (Sentence Bag of Words)	37
CBOW (Continuous Bag of Words)	37
Two-Pass Architecture	38
Configuration (Makefile variables)	38
Memory Considerations	38
Phase 2: Network Training (<code>BasicModel.py</code>)	38
Masked Prediction Modes	39
<trainEmbedding>: Embedding Update Mode	39
Gradient Flow Through Codebook	39
JOINT: Single Combined Loss	40
Why MSE over Embeddings (not Cross-Entropy over Vocabulary)	40
Training Loop	40
SBOW vs CBOW	41
Three-File Architecture	41
Embedding Space Geometry	41
Profiling	42
Abstract	42
Background	43
Part I: What Machine Minds Are — Weight Ergodicity	43
Weights as Ergodic Samples	43
Temperature and Annealing	43
Advantages of Ergodic Weights	44
Part II: What Machine Minds Feel — Network Invertibility	44
Bidirectional Perception	44
Symbols, Worldlines, and Karmic Inertia	44
Implementation	45
Part III: What Machine Minds Know — Output Certainty	45
Certainty vs. Probability	45
Certainty-Weighted Loss	45
Knowing What You Do Not Know	46
Methods	46
Results	47
Discussion	47
Conclusion	48
References	48
XML Configuration Parameters	48
<architecture>	49
<architecture><data>	49
<architecture><training>	49
<InputSpace>	50
<PerceptualSpace>	50

<ConceptualSpace>	51
<SymbolicSpace>	51
<SyntacticSpace>	51
<OutputSpace>	52
InvertibleLinearLayer	52
<architecture><server>	52
Example: XOR Configuration	53
Example: Embedding / Chatbot Configuration	54
Installation and Usage	55
Prerequisites	55
Setup	55
Makefile Targets	55
Training	55
Models	55
Testing and Benchmarks	55
Documentation	56
Utilities	56
Makefile Variables	56
train.py Options	56
General	56
Remote Execution (SSH)	57
Environment Variables	57
Remote Training (ArborMini)	57
Workflow	57
SSH Key Setup	58
Quick Start	58

BasicModel

Basic Model of Cognition

The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Documentation

Document	Description
Architecture	Pipeline design, layer types, invertible LDU factorisation
Spaces	All six spaces: Input $\rightarrow$ Perceptual $\rightarrow$ Conceptual $\rightarrow$ Symbolic $\rightarrow$ Syntactic $\rightarrow$ Output
Ergodic	Gradient energy sensor, adaptive exploration, factor-level noise injection
Training	Two-phase training, SBOW embeddings, masked prediction modes
Params	Complete XML configuration reference
BasicModel	Cognitive science foundations
Language	Grammar, word encoding, symbolic and syntactic spaces
Logic	Subsymbolic and symbolic logic operations
MachineMinds	What machine minds are, feel, and know
Installation	Setup, Makefile targets, environment variables

Overview

BasicModel is a parameterized neural architecture with three independent levers:

- **Ergodic** — adaptive bias-variance control via a gradient energy sensor (not gradient descent)
- **Certainty** — per-neuron certainty tracking; neurons graduate from exploration to exploitation independently
- **Reversible** — bidirectional training: forward prediction + backward reconstruction in a single optimizer pass

Model configurations are specified in XML and can be compared side-by-side. See doc/Architecture.md for the full mathematical treatment.

Files

File	Description
bin/BasicModel.py	Main entry point: model factory, training loop, comparison plots, HTML report
bin/Model.py	Layer library: SigmaLayer, PiLayer, ErgodicLayer, LinearLayer, spaces
bin/embed.py	Word vector training: CBOW/SBOW with negative sampling, WordVectors (gensim-compatible .k
bin/SigmaPi.py	Standalone demo of the SigmaPi network solving XOR
data/	XML model configurations
doc/Architecture.md	Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor
doc/Params.md	Full XML parameter reference
doc/Training.md	Embedding pretraining, CBOW/SBOW, masked prediction, <trainEmbedding> modes
doc/Installation.md	Setup, Makefile targets, train.py options, remote training
test/	Unit tests

Quick Start

```
# Set up virtual environment
make install

# Run a single model
make simple      # data/simple.xml
make ergodic     # data/ergodic.xml

# Compare two models side-by-side
make compare     # defaults: data/simple.xml vs data/ergodic-only.xml
make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test

# Generate PDF documentation
make doc_pdf
```

XML Configuration

Models are configured via XML files in data/. Training and data parameters live in nested sub-elements:

```
<model>
  <architecture>
    <modelType>simple</modelType>  <!-- simple / lm / embedding -->
    <ergodic>false</ergodic>
    <certainty>false</certainty>
```

```

<reconstruct>NONE</reconstruct> <!-- NONE / symbols / output / both -->
<maskedPrediction>NONE</maskedPrediction>

<data>
  <dataset>xor</dataset>          <!-- xor / mnist / text / ... -->
</data>

<training>
  <numTrials>1</numTrials>
  <numEpochs>20</numEpochs>
  <batchSize>10</batchSize>
  <learningRate>0.001</learningRate>
  <weightsPath>output/BasicModel.ckpt</weightsPath>
  <autoload>true</autoload>
  <autosave>>false</autosave>
</training>
</architecture>

<InputSpace> ... </InputSpace>
<PerceptualSpace> ... </PerceptualSpace>
<ConceptualSpace> ... </ConceptualSpace>
<SymbolicSpace> ... </SymbolicSpace>
<OutputSpace> ... </OutputSpace>
</model>

```

See doc/Params.md for the full parameter reference.

Output

Each run produces an HTML report (timestamped in `output/`) containing:

- **Error per Epoch** — training and test loss curves
- **Accuracy per Digit** — per-class accuracy breakdown

In compare mode, additional overlay plots show combined loss and accuracy across models with color-coded legends.

Architecture

Overview

BasicModel is a bidirectional neural architecture organized as a pipeline of six **spaces**, each implementing a distinct representational transformation:

Forward: `InputSpace` → `PerceptualSpace` → `ConceptualSpace` → `SymbolicSpace` → `SyntacticSpace` → `OutputSpace`
Reverse: `OutputSpace` → `SyntacticSpace` → `SymbolicSpace` → `ConceptualSpace` → `PerceptualSpace` → `InputSpace`

The forward pass transforms raw input into predictions. The reverse pass reconstructs the original input from the symbolic representation. Both directions are trained simultaneously with a single optimizer minimizing a combined loss:

$$\text{totalLoss} = (1 - \text{reconRatio}) * \text{outputLoss} + \text{reconRatio} * \text{reconstructionLoss}$$

Spaces

Space	Role	Layer Type	Parameters
InputSpace	Lifts raw data into working dimensionality	LiftingLayer	nActive, nDim, nV
PerceptualSpace	Multiplicative feature extraction	PiLayer	nActive, nDim, nV
ConceptualSpace	Additive abstraction	SigmaLayer	nActive, nDim, nV
SymbolicSpace	Discrete activation bottleneck	InvertibleLinearLayer + Codebook	nActive, nVectors,
SyntacticSpace	Binary derivation tree (CNF grammar)	Grammar + WordEncoding	nActive, nDim, nV
OutputSpace	Final prediction	LinearLayer	nActive, nDim, nV

Dimensions (`nDim`) are not passed to Space constructors; each subclass reads its content dimensionality from `TheObjectEncoding` (e.g., `TheObjectEncoding.perceptDim` for `PerceptualSpace`). Codebook sizes (`nVectors`) are likewise stored on `TheObjectEncoding` and may differ from the active count (`nActive`); the factory validates `nVectors >= nActive` for every space.

Layer selection depends on `<reconstruct>` and `invertible`:

1. **reconstruct=NONE**: Use non-invertible layers (`PiLayer`, `SigmaLayer`) for the forward pass only. No reverse pipeline is created.
2. **reconstruct=<any> + invertible**: A single invertible layer (`InvertiblePiLayer`, `InvertibleSigmaLayer`) serves both directions, sharing weights.
3. **reconstruct=<any> + not invertible**: Two layers with separate weights — call `forward()` on one and `reverse()` on the other. This avoids the expressivity limitation where a non-invertible layer's forward pass cannot represent the inverse of another (e.g., `PiLayer`'s product structure is not closed under inversion). **Note**: The reverse path uses a matrix pseudoinverse (`pinv`) which may be numerically unstable due to SVD convergence issues. Setting `<invertible>true</invertible>` avoids this by using shared-weight inversion instead.

Reconstruction Symbols

The symbolic bottleneck can lose information needed for reconstruction. For example, XOR maps 2 inputs to 1 output, but `XOR(0,0)=0` and `XOR(1,1)=0` are distinct inputs producing the same output. A single output symbol cannot reconstruct which input was presented.

To solve this, the `nSymbols` produced by `SymbolicSpace` are split:

- `nOutputSymbols = OutputSpace.nActive` — fed to `OutputSpace` for prediction
- `nReconSymbols = nSymbols - nOutputSymbols` — carried in `end_state` for reconstruction

This is essentially a skip connection through the symbolic bottleneck: an extra channel that preserves information lost in the output projection. Reconstruction symbols receive gradient only from reconstruction loss, never from output loss.

For XOR with `nSymbols=3`, `nOutput=1`: symbol 0 predicts the output; symbols 1-2 carry enough information to distinguish all 4 input patterns, enabling perfect reconstruction.

Single Optimizer with Overlapping Weight Spaces

A critical design choice: the forward and reverse passes share a **single Adam optimizer** that minimizes a combined loss:

```
totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss
```

This works because the forward and reverse weight spaces **partially overlap**. They are neither disjoint (which would allow independent optimizers) nor identical (which would create destructive interference). Instead, certain layers share weights between directions (e.g., shared embeddings, the symbolic bottleneck itself) while others are direction-specific (e.g., `pi1/pi2`, `sigma1/sigma2`, `linear1/linear2`).

The partial overlap means:

- **Shared weights** receive gradient from *both* output and reconstruction loss, learning representations useful in both directions simultaneously.
- **Direction-specific weights** receive gradient from only their respective loss, specializing freely without interference.
- **No ping-pong**: separate optimizers on overlapping parameters would pull weights in alternating, conflicting directions each step. A single optimizer sees the combined gradient and finds a consistent descent direction.

This resolves the fundamental tension between forward prediction and backward reconstruction in bidirectional architectures — a problem first identified in A.M. Rogers, T.T. Shannon, and G.G. Lendaris, “A comparison of DHP based antecedent parameter tuning strategies for fuzzy control,” *Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, Vancouver, BC, Canada, 2001, pp. 580-585, doi: 10.1109/NAFIPS.2001.944317. See also `doc/research/` for a local copy.

When `invertible=true`, the overlap is total: a single invertible layer serves both directions, and its weights receive the full combined gradient. The single-optimizer approach handles this case naturally.

Training Loop

Training uses the single Adam optimizer with persistent state (momentum/variance accumulate across epochs):

1. Forward pass: input $\rightarrow$ prediction + `end_state`
2. Compute `outputLoss` from prediction vs. target
3. Reverse pass: `end_state` $\rightarrow$ reconstructed input
4. Compute `reconstructionLoss` from reconstruction vs. `end_state`
5. Backpropagate `totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss`
6. If ergodic: run `paramUpdate()` (gradient energy sensor updates alpha)
7. Optimizer step (embedding params excluded when `trainEmbedding` is NONE or ARLM)
8. If `trainEmbedding` is CBOW, SBOW, or BOTH: run embedding update step on same sentence

Alpha annealing (ergodic mode): the code-level exploration parameter starts at 1.0 (full exploration) and decays to 0.0 (full exploitation) within the first 5% of epochs via `alpha = max(0, 1 - epoch / warmup)` where `warmup = numEpochs // 20`. Note: the code convention (`alpha=1` means explore) is the inverse of the ergodic math convention (`alpha=0` means explore); layers translate between them internally.

See `Params.md` for all XML configuration parameters. See `Training.md` for embedding pretraining, SBOW, masked prediction modes, and the `<trainEmbedding>` control.

Three-File Architecture

Model behaviour is partitioned across three independent artifacts:

File	Contents	Managed by
XML config (e.g. <code>BasicModel.xml</code>)	Architecture, hyperparameters, paths	Hand-edited
Embedding artifact (e.g. <code>BasicModel.kv</code>)	Word vectors (codebook)	Phase 1: <code>embed.py</code>
Weights checkpoint (e.g. <code>BasicModel.ckpt</code>)	Model layer parameters (excludes embeddings)	Phase 2: <code>BasicModel.py</code>

The `save_weights()` method explicitly filters out embedding parameters (`_emb.weight`) from the checkpoint. Embedding vectors live in the `.kv` artifact and are loaded separately. This separation allows:

- Retraining embeddings without touching model weights
- Retraining model layers with frozen embeddings
- Swapping codebooks between models that share the same architecture

Language System

The symbolic and syntactic spaces implement a binary deep-structure grammar in Chomsky Normal Form. SymbolicSpace maps continuous concept activations to a discrete one-hot encoding via an invertible layer and codebook. SyntacticSpace generates a derivation tree from the active symbols, stored as word tuples (batch, vector, rule).

See Language.md for the full grammar, word encoding, and open implementation questions about differentiable tree structure and rule operations.

Sigma and Pi Layers

Given a weight matrix ($W \in \mathbb{R}^{m \times n}$) and input vector ($x \in \mathbb{R}^n$), the output vector ($y \in \mathbb{R}^m$) is computed as:

For the Sigma layer:

$$y_j = Wx + b$$
$$y_j = b_j + \sum_{i=1}^n W_{ji}x_i$$

For the Pi layer:

$$y_j = b_j \cdot \prod_{i=1}^n (1 + W_{ji}x_i)$$

This is not invertible, but if we are allowed twice as many outputs as inputs, we may: do inversion as:

1. Define:

$$s_j := \sum_i \tanh^{-1}(w_{ji}x_i) = Wx$$

2. In forward pass, use:

$$y_j = b_j \cdot \prod_i (1 - \tanh(w_{ji}x_i)), \quad z_j = b_j \cdot \prod_i (1 + \tanh(w_{ji}x_i))$$

3. In reverse pass, compute:

$$\gamma_j = \frac{1}{2} \log \left(\frac{z_j}{y_j} \right) = \sum_i \tanh^{-1}(\tanh(w_{ji}x_i)) = \sum_i w_{ji}x_i \Rightarrow \gamma = Wx$$

4. Finally:

$$x = W^{-1}\gamma$$

Dimensionality Constraints

- The output dimensionality of the input layer must be equal to the output dimensionality of the perceptual layer, since the conceptual layer operates on both.
 - The input dimensionality of the symbolic layer must be equal to the input dimensionality of the perceptual layer, since they both operate on the output of the conceptual layer.
 - The input dimensionality of the output layer must be equal to the sum of the output dimensionalities of the symbolic layers.
-

Pi Layer Derivation

$$\begin{aligned}
e^{y_j} &= e^{b_j} + \sum_{i=1}^n e^{1+W_{ji}x_i} \\
\log(e^{y_j}) &= \log(e^{b_j} + \sum_{i=1}^n e^{1+W_{ji}x_i}) \\
\log(e^{y_j}) &= \log(e^{b_j}) \cdot \log\left(\sum_{i=1}^n e^{1+W_{ji}x_i}\right) \\
y_j &= b_j \cdot \prod_{i=1}^n \log(e^{1+W_{ji}x_i}) \\
y_j &= b_j \cdot \prod_{i=1}^n (1 + W_{ji}x_i)
\end{aligned}$$

Now, is there a way to write $e^{1+W_{ji}x_i}$ as a matrix product, $W_{ji}x_i$? If we write W and x in the log domain first, we have:

Invertible Linear Layer (LDU)

InvertibleLinearLayer — LDU Factorisation

The core invertible linear primitive factors the weight matrix W as:

$$W = L @ D_embed @ U$$

where:

- $L \in \mathbb{R}^{nIn \times nIn}$: unit lower-triangular matrix (diagonal fixed at 1). Stored as `raw_L`; the strict lower triangle is extracted and the diagonal is forced to 1 at each forward call.
- D : diagonal vector of length `rank = min(nIn, nOut)`, embedded into a rectangular `[nIn, nOut]` matrix `D_embed` by zero-padding.
- $U \in \mathbb{R}^{nOut \times nOut}$: unit upper-triangular matrix (diagonal fixed at 1). Stored as `raw_U`; the strict upper triangle is extracted symmetrically.

Exact inverse via triangular solves.

$$W^{-1} = U^{-1} @ D^{-1} @ L^{-1}$$

Each factor is inverted by a triangular solve (`torch.linalg.solve_triangular`). No SVD is required, and the inverse is exact as long as all diagonal entries of D are nonzero.

Parameter count. $nIn^2 + rank + nOut^2$ total parameters. Initialized at $L = I$, $d = 1$, $U = I$ so that the initial map is the identity.

Ergodic Noise Injection (Factor-Level)

Rather than the matrix-level blend $W_eff = b \cdot W + t \cdot N$ (which destroys the LDU structure and makes the inverse approximate), noise is injected directly into the raw parameters of each factor before the triangular structure is extracted:

$$\begin{aligned}
L_eff &= I + \text{strict_lower}(\text{raw_L} + t \cdot \text{noise_raw_L}) \\
U_eff &= I + \text{strict_upper}(\text{raw_U} + t \cdot \text{noise_raw_U}) \\
d_eff &= b \cdot d_effective + t \cdot \text{noise_d} \\
W_eff &= L_eff @ D_eff_embed @ U_eff
\end{aligned}$$

Because `W_eff` is still in LDU form, its exact inverse is always available by the same triangular solves — no approximation, regardless of the noise level.

`stable=True`

When `stable=True`, `_d_effective()` clamps each diagonal entry `d_i` to magnitude `[eps, 1]` with sign preserved before the ergodic blend:

```
d_clamped_i = sign(d_i) * clamp(|d_i|, eps, 1)
d_eff = b · d_clamped + t · noise_d
```

This keeps `d_eff` bounded away from zero, preventing `W_eff` from becoming singular. `stable=True` is the only place the stability constraint is enforced; no additional clamp on `d_eff` is needed.

`noise_d`

Noise for the diagonal factor is sampled with $|d_i| \in [\text{eps}, 1]$ and random sign, so the noise factor is itself always invertible. This ensures that even at full temperature (`b=0`, `t=1`) the effective matrix is still well-conditioned.

naive Flag

naive	Forward path	Reverse path
False (default)	Apply L, D, U sequentially to <code>x</code> ; backprop through each factor separately	Triangular solves: U^{-1} , D^{-1} ,
True	Materialise <code>W_eff</code> as a dense matrix; apply <code>W_eff @ x</code>	<code>pinv(W_eff) @ y</code>

The `naive=False` path never materialises `W_eff` as a full matrix, saving memory and allowing each factor’s gradients to flow independently.

Noise Lifecycle in Ergodic Mode

Noise buffers (`noise_raw_L`, `noise_raw_U`, `noise_d`) are:

1. **Resampled at the start of every ergodic forward()** — new noise is drawn before constructing `W_eff`.
2. **Resampled again at the end of every ergodic reverse()** — fresh noise is drawn after the reverse computation completes.

The window between the two calls is exactly when `reverse()` needs to reconstruct the same `L_eff`, `U_eff`, `d_eff` that `forward()` used. Because the buffers are not resampled during that window, `reverse()` reads the stored buffers directly — no explicit caching of `W_eff` is required.

Class Renames

The LDU layer supersedes the previous SVD-based implementation:

Old name	New name	Status
<code>LULayer</code>	<code>InvertibleLinearLayer</code>	Renamed
<code>InvertibleLinearLayer (SVD)</code>	—	Removed
<code>InvertibleSigmaLayer</code>	<code>SigmaLayer(invertible=True)</code>	Merged
<code>InvertiblePiLayer</code>	<code>PiLayer(invertible=True)</code>	Merged

Ergodic Exploration

The ergodic bias-variance control system is documented in Ergodic.md.

Basic Model

The Basic Model of Cognition is a neural network that consists of four spaces: real space, perceptual space, conceptual space, and symbolic space.

Model Entities:

Entity	Meaning
$x, \hat{x}, X$	Actual and predicted input (experiences), input space
$y, \hat{y}, Y$	Actual and predicted output (actions), output space
p, P	Percepts, perceptual space
c, C	Concepts, conceptual space
s, S	Symbols, symbolic space
W_e	Experiencing, experiential weights
W_a	Acting, action weights
W_k	Knowing, knowledge weights
W_t	Thinking, thought weights

Background

Much of this work is based on the Basic Model of Cognition described briefly at cognitivesciencesociety.org/framework-cognition/. More details are provided in the book *The Whole Part*.

Inputs and Outputs

Inputs and outputs derive from a real space that is ineffable. This essay merely characterizes them relative to other things.

The adaptable parameters of a mind are initially zero, a state known as *tabula rosa*. Learning is initiated by random perturbations of a map relative to its territory, and the degree of that exploration is referred to as *temperature*.

Input are normalized and lie on a hypersphere. They are nonnegative because negative entities do not exist. To paraphrase Saint Augustine, *negative is the privation of the positive*.

Outputs and Inputs are dehumanizing terms: the world is a living world, so it helps to call outputs “actions” and inputs “experiences”. When minds perceive and act within the world, their mistakes are called expectation errors and performance errors.

Action is a function of perceptual space and our expectation, where that space is often filtered by conceptual projections.

Performance errors are defined relative to action, desired effect, and exhaustion.

Expectation errors are defined relative to perception and perception-as-reconstructed.

To accommodate an input context window which changes, the input is augmented with relative spatiotemporal encoding by extending its dimensionality. While much of the network can treat these as simply additional features of the input, the attention mechanism must deal with them explicitly since attention is directed within space and time (i.e. it treats the what and where pathways separately).

Perceptual Space

Perceptual spaces are composed of perceptual prototype vectors, or *percepts*.¹

¹Perceptual space is known as “embedding space” in the current literature on transformer neural net architectures.

Perceptual spaces map onto reality in a straightforward way, at least in so far as we perceive correctly. They are bidirectional, although not necessarily symmetric. Technically, they are real similarity spaces with arbitrarily high dimensionality, where logical negation is not defined.

Percepts that do not correspond to objects are *hallucinations*.

Percepts generalize a perceptual space by forming a Voronoi tessellation using prototype and/or exemplar vectors. They are related to one another in virtue of a connection which defines a Self Organizing Map. Percepts operate in parallel. Insofar as categories are concerned, they form an embodiment of prototype theory.

The perceptual error function is defined by the mismatch between perception and existing percepts, where that mismatch may exert an influence on the world.

Perceptual accuracy is a measure of the distance between inputs and prototype vectors. That distance is similar to cosine similarity, although to the extent we don't know a perceptual well, it is a simple distance measure in the range (0-1).

Attraction and aversion within a perceptual space manifest as weighted prototype vectors: attraction is a pole at the percepts's location, and aversion is a zero at that location. As a result, they warp perceptual space, making us more or less likely to perceive things that we love or hate.

Perfectly recognized percepts lie on the unit hypersphere within conceptual space.

Two percepts can be joined into one concept without loss of characterization.

Conceptual Space

Concepts create boundaries in conceptual space, a space which is grounded in the support vectors of perception. By creating boundaries, they support logical negation. Due to this relativity, they may refer to negative entities which may or may not exist on perceptual or objective levels.

Conceptual spaces are capable of representing negative entities.

Concepts are references that do not exist in the space in which the percepts are defined, but in the space formed by the collections of perceptual activations. Thus, the space of concepts can not be directly overlaid on the space of percepts, because it occurs at a different epistemic level, although conceptual space can be projected onto perceptual space.

Concepts that do not correspond to percepts are *false*.

Concepts collect percepts into fuzzy sets. Percepts are weighted by a degree of membership, which allows concepts to perform several logical operations such as and, or, and not. They can be seen either as linear separating hyperplanes which partition conceptual space into two parts, or as sets which are defined as a linear combination of the percept activations.

Concepts are relative: they are meaningful only insofar as they create a partition of a meaningful space.

As the concepts of a mind approach certainty, the slope of the decision boundary becomes infinite.

There is a single state of unknowing which is common to all knowledge vectors, which does not categorize space because the slope of the activation function is zero.

It is also the case that knowledge vectors of length zero are undirected, which seems like a state of unknowing which is slightly different than slightly different the measure of certainty. Informally, the concept must be learned before its certainty can be assessed.

Thus, conceptual spaces are similarity spaces that are partitioned by decision boundaries that impose positivity and negativity on a hypersphere of knowing.

Percepts are incorporated into meronomies through the use of concepts, which create transitive part hierarchies.

When symbols are so incorporated, the referential nature of symbols creates intransitive part hierarchies.

Within conceptual space, attraction and aversion to concepts can be expressed as a hyperplane bias (i.e. the tendency to partition reality in a biased way).

Within conceptual space, certainty is expressed as the slope of the activation function. Is it tuned after the other parameters have ceased to change as a result of correct categorization.

Certainty (or clarity) can also be grounded in symbolic space, where symbolic support vectors are taken as *a priori* truth (or at least learned with some certainty reflected in their nonzero magnitude).

Concepts can be split into two precepts without loss of characterization.

Symbolic Space

Symbolic Spaces are composed of percepts that are references to concepts. They are therefore a subtype of perceptual spaces that map onto conceptual spaces.

Because symbols do not have any spatial context, they exist as zero-dimensional points within perceptual space. If we don't have feeling receptors in the brain, there is no positional encoding for this layer.

In virtue of that representation, symbols can be conceptualized and collected into sets. The creation of sets of symbols, since they have no inherent ordering, creates equivalence classes.

In humans, perceptual space is projected onto a 3D subspace within the cerebellum before symbolic encoding.

Symbols are independent, permanent, and integral (at least when interpreted *as* references, even though they ultimately exist within a dependent, temporary, and continuous space).

The top-down, serial activation of a symbolic space differs from the bottom-up activation of perceptual spaces because it casts shadows.

The formation of symbols increases the dimensionality of conceptual space (as would an outer product).

The formation of concepts from percepts decreases the dimensionality of perceptual space (as would an inner product).

Symbols are zero-dimensional entities, or epistemological points. As references, symbols form a discrete perceptual space and can be represented as a binary array.

The participation of symbols in multiple sets creates structural relations which takes the place of positional information.²

Symbols are erroneous if their associated concept is not true.

Reasoning

Reasoning within symbolic spaces involves computing with parts and wholes. This may be seen as using Venn diagrams to perform computations on a space, which results in ability to perform Bayesian-type statistics dynamically.

Those probabilities, once learned, can be stored as unidirectional connection weights between concepts, perhaps in conjunction with a part-whole structure imposed on those concepts by their corresponding symbols.

Symmetric Perception: the Single Layer Linear Case

- A symmetric perception network predicts both input and output. By taking advantage of functions that may be surjective in two directions, it is able to compute a bijective mapping.
- The term “*symmetric perception*” refers to the fact that perception is bidirectional: it is an interaction in which we see the world and in which the world sees us. The latter fact is often forgotten from an egocentric point of view; culturally, as Edward has observed, perception is seen as a passive process. In

²For example, $\{0,1\}$ and $\{0,1\}$, if those sets are orthogonal to one another, pose no constraint on the location of the 1 within the 2x2 grid that they partition. However, $\{0,1,1\}$ and $\{0,1\}$ partition a 3x2 space and simultaneously impose the constraint that two 1's must lie on the same row. This is a fact which is not present in either the row set or the column set.

the context of neural networks, which often serve as models of the mind, this egocentric view continues: functions are computed that map inputs onto outputs, and the influence of the outputs on the inputs is forgotten. This essay explores the technical aspects of symmetric perception as well as several guiding principles for humanistic mental models.

Symmetric Perception, in the general case, finds a function $f(x)$ which minimizes the MSE with respect to the following:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & f^{-1}(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

In the linear case, $f(x)$ becomes a single invertible matrix A . In general, the mapping from input to output may not be linear, and the inverse may not exist, in which case the previous set of input/output equations is written as:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & g(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

This problem bears some resemblance to the standard regression problem $Ax = b$, except that x is known and we wish to find the set of points A (or weights) that map one linear space to another. So the familiar Linear Least-Squares solution to the equation $x = (A^T A)^{-1} A^T b$ becomes an attempt to find a matrix A such that:

$$Ax = b \text{ and } A^{-1}b = x$$

This solution is seen as the “best” solution by neural networks that wish to determine a weight matrix A that maps from x onto b . That much is shared between the two problems, although there are nonlinearities in the general neural network case which enable a much better fit. This solution is optimal in the sense that it minimizes the Sum of Squared Errors, where the errors are defined as the residuals when b is projected into the column space of A .

However, the principle of retrocausality suggests an additional objective; namely, we wish to create a matrix such that it is optimal both for the transformation of the input to the output and the output to the input.

In general, we want a matrix A such that minimizes reconstruction loss over both $Ax = b$ and $A^{-1}b = x$. Since we are simultaneously minimizing the error vectors of each, and since the column space of a matrix is identical to the column space of its inverse, we will end up with a matrix A that contains the vectors x and b within both its column space and row space (which is necessary because we are seeking a bijective function).

As a first approximation, we might write A as a product of two matrices, each of which performs a perfect surjection: $A \cdot A^{-1} = I$

$$\left(\frac{b}{2} * x^{-1}\right) \cdot \left(\frac{x}{2} * b^{-1}\right) = I$$

However, this will still be only an approximate solution, since it is unlikely that a somewhat arbitrary combination of both matrices will result in the correct orthogonal matrix. To find the best matrix A involves either an iterative solution as would be performed by NN learning, ping-pong style, or (theoretically?) via multiple round trips consisting of Gram-Schmidt orthogonalization with respect to the matrices A and A^{-1} .

Symmetric Perception: the Single Layer Nonlinear Case

If $f(x)$ and $g(x)$ share a common weight matrix W , we may write the equations for symmetric perception as:

Forward computation and gradient:

$$\hat{y} = f(g^{-1}(x)W)$$

$$e_y = \frac{1}{2}(\hat{y} - y)^2$$

$$\frac{\partial e_y}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial \hat{y}}{\partial W} = \delta_f(W^\top \cdot g^{-1}(x))$$

$$\frac{\partial W}{\partial x} = \delta_g(x)$$

$$\Delta W_y = \eta \frac{\partial e_y}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W} g^{-1}(x)$$

$$\Delta W_y = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)$$

Reverse computation and gradient:

$$\hat{x} = g(W^\top f^{-1}(y))$$

$$e_x = \frac{1}{2}(\hat{x} - x)^2$$

$$\frac{\partial e_x}{\partial \hat{x}} = \hat{x} - x$$

$$\frac{\partial \hat{x}}{\partial W} = \delta_g(W \cdot f^{-1}(y))$$

$$\frac{\partial W}{\partial y} = \delta_f(y)$$

$$\Delta W_x = \eta \frac{\partial e_x}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial W} f^{-1}(y)$$

$$\Delta W_x = \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Since the single matrix W is shared, we might combine the partial derivatives of the input and output errors via sum and set the gradient to zero to explore the solution space, which seems to minimize the cross-covariance between the input and output spaces:

$$\frac{\partial e_y}{\partial W} + \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) + \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

If we assume that the gradients never go to zero, then we have the sum of the partial derivatives with respect to the error going to zero when both $x = i$ and $y = o$. However, it may be the case that $xy - iy = ox - yx$,

which means that the simultaneous gradient descent is ping-ponging between the two cost functions. So a better cost function would be a multiplication of the two error partials, which is an intersection of the space that satisfies both equations. In that case, we can multiply the terms to get a better understanding of the error surface that we are minimizing with respect to the input and the output:

$$\frac{\partial e_y}{\partial W} \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x)(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Distance Metrics

$$d(x_1, x_2) = \frac{\|x_1\|^n \cdot \|x_2\|^n}{\|x_1 - x_2\|^{2n}}$$

Which can be written in the Z-plane as:

$$d(x_1, x_2) = \|x_1 - x_2\| \cdot \frac{(x - z_1)}{(x - z_2)}$$

Where z_1 is a zero and z_2 is a pole.

Exploration/Exploitation as a function of temperature:

$$y = x \cdot \frac{(W + rt)}{\|W\|}$$

Where:

- x, y are the input and output vectors
- W is the weight matrix
- r is a random vector
- t represents temperature

Transfer Functions

By using rotation matrices with a tunable parameter Θ before and after the projection from one space to the other, and by restricting the projection matrix to be diagonal, it is possible to restrict the solution space to an eigendecomposition of the orthonormal basis that maps input to output (i.e. it calculates the SVD of the matrix). For the 2D case, this can be written:

$$y = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 \\ \sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 \\ \sin \theta_2 & \cos \theta_2 \end{bmatrix} x$$

It may also be possible to use a nontunable scalar function as the transfer function and not restricting the matrix to be diagonal, which is approximately how neural networks apply nonlinearities to the sum of products by weight matrices.

$$f(x) = \sin(x)$$

$$g(x) = \cos(x)$$

$$f^{-1}(x) = \arcsin(x)$$

$$g^{-1}(x) = \arccos(x)$$

$$\partial_f(x) = \cos(x)$$

$$\partial_g(x) = \sin(x)$$

$$\partial_{f^{-1}}(x) = \frac{1}{\cos(\arcsin(x))}$$

$$\partial_{g^{-1}}(x) = \frac{1}{\sin(\arccos(x))}$$

If we write the forward equations and gradients of $f()$ and $g()$ using these functions, we end up with the following formulation:

$$\hat{y} = \sin(\cos^{-1}(x)W)$$

$$\hat{x} = \cos(W^\top \sin^{-1}(y))$$

$$\Delta W_y = \eta(\hat{y} - y) \cos(W^\top \cdot \cos^{-1}(x)) \cos^{-1}(x)$$

$$\Delta W_x = \eta(\hat{x} - x) \sin(W \cdot \sin^{-1}(y)) \sin^{-1}(y)$$

The important thing to notice is that the reconstruction of the reverse perception and the calculation of the gradient of forward perception involve common terms, and may be simplified:

$$\hat{y} = f(g^{-1}(x)W)$$

$$\hat{x} = g(W^\top f^{-1}(y))$$

$$\Delta W_y = \eta(\hat{y} - y) \delta_f(W^\top \cdot g^{-1}(x)) g^{-1}(x)$$

$$\Delta W_x = \eta(\hat{x} - x) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y)$$

$$\Delta W_y = \eta(f(g^{-1}(x)W) - y) \delta_f(W^\top \cdot g^{-1}(x)) g^{-1}(x)$$

$$\Delta W_x = \eta(g(W^\top f^{-1}(y)) - x) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y)$$

$$\Delta W_y \Delta W_x = \eta(f(g^{-1}(x)W) - y) \delta_f(W^\top \cdot g^{-1}(x)) g^{-1}(x) \cdot (g(W^\top f^{-1}(y)) - x) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y)$$

If we take the product of both:

$$\Delta W_y \Delta W_x = \eta(\hat{y} - y) \delta_f(W^\top \cdot g^{-1}(x)) g^{-1}(x) \cdot (\hat{x} - x) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y)$$

$$\Delta W_y \Delta W_x = \eta(\hat{x} - x) (\hat{y} - y) \delta_f(W^\top \cdot g^{-1}(x)) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y) g^{-1}(x)$$

Spaces

Overview

BasicModel is organized as a pipeline of six **spaces**, each performing a distinct representational transformation. Data flows forward from raw input to task output; the reverse pass reconstructs the original input from the symbolic representation.

Forward: `InputSpace` $\rightarrow$ `PerceptualSpace` $\rightarrow$ `ConceptualSpace` $\rightarrow$ `SymbolicSpace` $\rightarrow$ `SyntacticSpace` $\rightarrow$ `OutputSpace`
Reverse: `OutputSpace` $\rightarrow$ `SyntacticSpace` $\rightarrow$ `SymbolicSpace` $\rightarrow$ `ConceptualSpace` $\rightarrow$ `PerceptualSpace` $\rightarrow$ `InputSpace`

Base Class: Space

All spaces inherit from a common **Space** base that manages the shared infrastructure:

Shape management. Each space tracks `inputShape` and `outputShape` as `[nObjects, nDim]` tuples, where `nObjects` is the active count and `nDim` is the per-object dimensionality. Dimensions are not passed as constructor arguments; each subclass reads them from `TheObjectEncoding` (e.g. `TheObjectEncoding.perceptDim` for `PerceptualSpace`).

Codebook / VQ quantization. When `nVectors` > `nActive`, the space maintains a codebook of `nVectors` candidate vectors and selects the `nActive` most activated ones via top-k selection. This is the vector-quantization bottleneck.

Reshape flag. When the input and output object counts differ, the reshape flag flattens the `[B, nObj, nDim]` tensor to `[B, nObj * nDim]` before passing it to the next space, then restores the object structure on the way back.

Attention. Each space may optionally include an attention mechanism (`hasAttention=true`) that reweights objects before the main transformation.

set_sigma propagation. When ergodic mode is active, `set_sigma()` is forwarded from the top-level model down through every space and into every child layer, so the per-neuron exploration level is kept consistent across the entire pipeline.

InputSpace

Role. Receives the raw source buffer from `Data()` and lifts it into the model’s internal working dimensionality.

Text mode — forward. Delegates tokenization to `Lex`, which produces a span table of (`start`, `end`, `type`) entries — one span per token. Each span is converted to a vector with two components:

- **nWhat** dimensions — token content, encoded via `Basis` / `Codebook` (the word embedding lookup).
- **nWhere** dimensions — positional information derived from the character offset.

The result is a `[nActive, nWhat + nWhere]` tensor representing the tokenized sentence.

Text mode — reverse. Reconstructs the source string from the latent state by inverting the span encoding: each vector is mapped back to its nearest codebook entry (word embedding), then spans are reassembled into characters using the stored offset table.

Numeric mode. Tensor data is passed through unchanged; the `LiftingLayer` projects the native input dimension (e.g. 784 for MNIST) to the model's working dimensionality `nDim`.

Key parameters.

Parameter	Description
<code>nActive</code>	Sequence length: maximum tokens per input
<code>nDim</code>	Output dimensionality per vector (set on <code>TheObjectEncoding</code>)
<code>nWhere</code>	Positional dimensions appended to each token vector
<code>nWhen</code>	Temporal dimensions appended to each token vector
<code>lexer</code>	Tokenization mode: "word" or "sentence"
<code>quantized</code>	Whether input values are discrete

Layer. `LiftingLayer` — bridges native input dimension to `nDim`.

Invertibility. `InputSpace` is always non-invertible in the strict sense; the reverse path is a separate reconstruction procedure using the span table, not a matrix inverse.

PerceptualSpace

Role. Transforms raw input vectors into perceptual representations via multiplicative interactions. Models prototype-based feature detection: each percept is a product of local input features.

Forward operation.

$$y_j = b_j * \prod_i (1 + W_{ji} * x_i)$$

Multiplicative structure allows the layer to detect conjunctions of features.

Reverse operation (`invertible=True`). A single `PiLayer(invertible=True)` is shared for both directions. The forward pass outputs interleaved (`log_y`, `log_z`) pairs, where:

$$y_j = b_j * \prod_i (1 - \tanh(W_{ji} * x_i))$$

$$z_j = b_j * \prod_i (1 + \tanh(W_{ji} * x_i))$$

The reverse pass recovers $\gamma = W x$ from $\gamma_j = 0.5 * \log(z_j / y_j)$, then solves for $x = W^{-1} * \gamma$ using `InvertibleLinearLayer`.

Reverse operation (`invertible=False`, `reversible=True`). Two separate `PiLayer` instances — `pi1` for forward, `pi2` for reverse — each with independent weights. The reverse layer uses a pseudoinverse to approximately invert the forward mapping.

`passThrough=True`. Identity: input passes through unchanged, no `Pi` computation.

Key parameters.

Parameter	Description
<code>nActive</code>	Number of active perceptual vectors
<code>nVectors</code>	Codebook size; enables VQ when $> nActive$
<code>invertible</code>	True: shared invertible layer; False: separate <code>pi1/pi2</code>
<code>passThrough</code>	Skip perceptual processing entirely
<code>hasAttention</code>	Enable attention reweighting

Layer. `PiLayer` (one or two instances depending on `invertible`).

Invertibility. `invertible=True`: shared layer, exact inverse. `invertible=False`: separate layers, approximate pseudoinverse in reverse.

ConceptualSpace

Role. Transforms perceptual vectors into abstract concepts via additive linear operations. Models conceptual hyperplanes that partition perceptual space.

Forward operation.

$$y_j = \tanh(W x + b)$$

The tanh nonlinearity squashes activations to $[-1, +1]$, giving each concept a graded membership value.

Reverse operation (`invertible=True`). A single `SigmaLayer(invertible=True)` shared for both directions. Exact inverse via:

$$\begin{aligned} \text{pre_tanh} &= \text{atanh}(y) \\ x &= W^{-1} * (\text{pre_tanh} - b) \end{aligned}$$

The matrix inverse W^{-1} is computed via `InvertibleLinearLayer` (LDU factorisation), so no SVD is required and the inverse is exact.

Reverse operation (`invertible=False`, `reversible=True`). Two separate `SigmaLayer` instances — `sigma1` for forward, `sigma2` for reverse.

Key parameters.

Parameter	Description
<code>nActive</code>	Number of active concept vectors
<code>nVectors</code>	Codebook size
<code>invertible</code>	True: shared invertible layer; False: separate <code>sigma1/sigma2</code>
<code>hasAttention</code>	Enable attention
<code>hasNorm</code>	Enable layer normalization

Layer. `SigmaLayer` (one or two instances depending on `invertible`).

Invertibility. `invertible=True`: exact inverse via $\text{atanh} + W^{-1}$. `invertible=False`: separate layers with independent weights.

SymbolicSpace

Role. Converts continuous concept activations into a discrete set of active symbols. This is the information bottleneck of the pipeline: rich perceptual–conceptual representations are compressed into a sparse activation pattern over a codebook of symbol prototypes. Symbols are **zero-dimensional** entities — pure activation scalars, not vectors.

See `Language.md` for the full language system design.

Forward operation (`quantized=True`).

1. Extract concept activation $[B, nConcepts]$ from the input subspace.
2. Map through `InvertibleLinearLayer(nConcepts, nSymbols)` to $[B, nSymbols]$.

3. Reshape to `[B, nSymbols, 1]` (each symbol is 1-dim) and pass through the codebook, which quantizes and produces a one-hot activation over codebook entries weighted by similarity.

The output is a one-hot encoding over the codebook. The codebook provides dense vectors for downstream spaces that require `[B, N, D]` tensors.

Forward operation (quantized=False). The invertible layer maps the activation; vectors pass through from the input subspace unchanged.

Reverse operation. The invertible layer's exact inverse maps `[B, nSymbols]` back to `[B, nConcepts]`, recovering the concept activation.

passThrough=True. Concept vectors and activation pass through unchanged.

Key parameters.

Parameter	Description
<code>nActive</code>	Total number of symbols (output + reconstruction symbols)
<code>nVectors</code>	Codebook size (= <code>nSymbols</code> when quantized)
<code>passThrough</code>	Skip symbolic processing entirely
<code>quantized</code>	Enable codebook quantization (required for one-hot output)

Layer. `InvertibleLinearLayer(nConcepts, nSymbols)` — maps between activation spaces of different lengths. Exact inverse via LDU factorisation.

Invertibility. Exactly invertible via the invertible layer's reverse.

SyntacticSpace

Role. Generates a binary derivation tree (deep structure) from the set of active symbols produced by `SymbolicSpace`. The derivation is a Chomsky Normal Form (CNF) grammar stored as word tuples (`batch`, `vector`, `rule`) on the output subspace.

See `Language.md` for the grammar, word encoding, and open questions about differentiable tree structure.

Forward operation.

1. Identify active symbol positions from the activation vector (nonzero entries).
2. For `N` active symbols per batch, generate a CNF derivation: `N-1` binary rules (randomly selected) + 1 terminal ($S \rightarrow W$).
3. Store the derivation as word tuples on the output subspace. Vectors and activation pass through unchanged.

Reverse operation.

1. Walk the word list: every (`batch`, `vector`, `rule`) entry marks that position as active.
2. Reconstruct the activation vector deterministically from the derivation.
3. Store the recovered activation (without recomputing from vector norms).

The round-trip `forward` $\rightarrow$ (`delete activation`) $\rightarrow$ `reverse` recovers the original active positions exactly.

Key parameters.

Parameter	Description
<code>nActive</code>	Number of active word vectors
<code>nDim</code>	Word vector dimensionality
<code>nVectors</code>	Codebook size (defaults to <code>nActive</code>)

Layer. No trainable parameters (rule selection is currently random).

Invertibility. Reverse deterministically recovers activation from the derivation tree.

OutputSpace

Role. Maps symbolic (or syntactic) vectors to task targets via a linear projection. This is the final prediction stage.

Forward operation. Linear projection from symbol dimensionality to output dimensionality:

$$y = W_{\text{out}} * x + b_{\text{out}}$$

Always uses `reshape=True` so the `[B, nSymbols, symbolDim]` tensor is flattened before projection.

Reverse operation. Projects output predictions back to symbol space via the pseudoinverse of `W_out`. In text mode, snaps each output vector to the nearest entry in the codebook (nearest-neighbour lookup) to recover a discrete token, then assembles tokens into a string.

getEmbeddedIO() override. `OutputSpace` overrides `getEmbeddedIO()` to return the raw target dimensions rather than the encoded dimensions. This ensures the loss is computed in the output vocabulary space, not the embedding space, regardless of any encoding overhead.

Text mode generation. In autoregressive language model mode, `OutputSpace` iterates token-by-token: at each step the predicted token vector is snapped to its nearest codebook entry, that entry is fed back as input, and generation continues until `maxResponseLength` is reached or an end-of-sequence token is produced.

Key parameters.

Parameter	Description
<code>nActive</code>	Number of output values (e.g. 1 for XOR, vocab size for LM)
<code>nDim</code>	Output vector dimensionality
<code>nVectors</code>	Codebook size (defaults to <code>nActive</code>)

Layer. `LinearLayer` with (`bias`, `temp`) support for ergodic mode. Always `reshape=True`.

Invertibility. Reverse uses pseudoinverse; not exactly invertible in general.

Language

Overview

The language system spans two spaces in the pipeline — **SymbolicSpace** and **SyntacticSpace** — together with a grammar (`TheGrammar`) and a word encoding (`WordEncoding`) that represent binary deep structure as derivation trees.

<code>ConceptualSpace</code>	$\rightarrow$	<code>SymbolicSpace</code>	$\rightarrow$	<code>SyntacticSpace</code>	$\rightarrow$	<code>OutputSpace</code>
<code>[B, nConcepts]</code>		<code>[B, nSymbols]</code>		<code>[B, nSymbols] +</code>		<code>[B, nOutput]</code>
activation		activation		word list		
		+ codebook		(derivation tree)		

The key insight: **symbols are discrete, one-dimensional entities**. They carry no content vectors (“what”) of their own. Instead, the activation pattern over a set of symbols *is* the representation, and the codebook provides a bridge back to dense vectors when downstream spaces require them.

Grammar

TheGrammar is a singleton instance of the **Grammar** class. It defines a context-free grammar in **Chomsky Normal Form** (CNF) — every rule is either binary (two non-terminal children) or terminal.

Rule	Production	Arity	Group
0	S	0 (leaf)	—
1	$S \rightarrow W$	0 (terminal)	conceptual
2	$S \rightarrow S \text{ AND } S$	2	conceptual
3	$S \rightarrow S \text{ OR } S$	2	conceptual
4	$S \rightarrow \text{NOT } S$	1 (unary)	conceptual
5	$S \rightarrow \text{NON } S$	1 (unary)	perceptual
6	$S \rightarrow S \text{ PART } S$	2	conceptual
7	$S \rightarrow S \text{ UNION } S$	2	perceptual
8	$S \rightarrow S \text{ INTERSECTION } S$	2	perceptual
9	$S \rightarrow S \text{ EQUALS } S$	2	perceptual

Rules are partitioned into two groups:

- **Grammar.conceptual()** $\rightarrow$ [1, 2, 3, 4, 6] — operations that manipulate abstract concepts (word reference, conjunction, disjunction, negation, parthood).
- **Grammar.perceptual()** $\rightarrow$ [5, 7, 8, 9] — operations on perceptual fields (non-affirming negation, set union, set intersection, equality).

This partition mirrors the two-layer logic system described in Logic.md: conceptual rules operate on the symbolic/scalar layer, perceptual rules operate on the subsymbolic/vector layer.

CNF and Unary Rules

Rules 4 (NOT S) and 5 (NON S) are unary — they have one non-terminal child, which breaks strict Chomsky Normal Form. Two design options remain open:

1. **Rewrite as binary** with an implicit identity/epsilon child: $S \rightarrow \text{NOT } S \ \varepsilon$. This preserves strict CNF at the cost of introducing null nodes.
2. **Allow unary rules** as a first-class concept — unary operators transform a single child’s representation without branching. The tree remains binary in the sense that all branching nodes have exactly two children; unary nodes are pass-through transformations.

Word Encoding

Each word is a 3-tuple (**batch**, **vector**, **rule**):

- **batch** — index into the batch dimension [0, B)
- **vector** — index into the activation vector [0, N)
- **rule** — grammar rule ID from **TheGrammar** [0, 10)

Words are stored as a Python list of tuples on the SubSpace (not muxed into the event tensor). The list constitutes a **derivation tree** in pre-order: the first entry is the root, and binary rules expand left-first.

The **WordEncoding** class validates tuple entries on construction. The **SubSpace** provides accessors: **add_word(batch, vector, rule)**, **set_words(list)**, **get_words()**.

SymbolicSpace

Role

Converts continuous concept activations into a discrete set of active symbols. This is the information bottleneck of the pipeline.

Forward Path

1. **Activation mapping.** Extract concept activation $[B, nConcepts]$ from the input subspace. Map through an `InvertibleLinearLayer(nConcepts, nSymbols)` to produce $[B, nSymbols]$ — a continuous activation in symbol space.
2. **Codebook quantization** (when `quantized=True`). The activation vector is reshaped to $[B, nSymbols, 1]$ (each symbol is a 1-dimensional scalar) and passed through the codebook. The codebook quantizes, computes top-k by similarity, and produces:
 - A one-hot-ish activation over codebook entries (similarity-weighted)
 - Dense vectors for downstream materialization

When `quantized=False`, the activation is stored directly on the subspace and the input vectors pass through unchanged.

3. **Output.** A SubSpace with symbol activation and (when quantized) codebook vectors. The output is a **one-hot encoding** over the codebook.

Reverse Path

1. Extract symbol activation $[B, nSymbols]$ from the input subspace.
2. Apply the **exact inverse** of the invertible layer to recover $[B, nConcepts]$.
3. Store the recovered concept activation on the output subspace, allowing ConceptualSpace to reconstruct its dense vectors.

Key Properties

- Symbols are **zero-dimensional** — they are pure activation scalars, not vectors.
- The invertible layer allows **different-length** activation spaces ($nConcepts \neq nSymbols$).
- When quantized, the codebook size equals `nSymbols` — each symbol *is* a codebook entry.
- The `discretize()` method (sigmoid + straight-through round to $\{0,1\}$) is available but currently unused; the codebook’s VQ serves as the discretizer.

SyntacticLayer

Architecture: Recursive Hybrid with Depth Embedding

`SyntacticLayer` is a weight-tied (recursive) derivation stack. A **single** shared derivation layer and rule head are applied repeatedly at each depth, with a learned depth embedding added to the hidden state so the shared weights can specialize by tree level.

$$\text{activation } [B, n_{\text{Symbols}}] \xrightarrow{\text{input_proj}} h \in \mathbb{R}^{B \times d_{\text{hidden}}}$$

For each depth $d = 0, 1, \dots, D$, using **shared weights**:

$$h \leftarrow h + \text{depth_embed}[d], \quad h \leftarrow \text{GELU}(\text{derivation_layer}(h)), \quad \text{logits}_d \leftarrow \text{rule_head}(h)$$

Parameters:

Component	Shape	Count (hidden_dim=256, num_rules=10)
input_proj	nSymbols $\times$ hidden_dim	varies
derivation_layer	hidden_dim $\times$ hidden_dim	65K
rule_head	hidden_dim $\times$ num_rules	2.5K
depth_embed	max_depth $\times$ hidden_dim	256 per depth level
Total (excluding input_proj)		$\sim 70\text{K} + 256 \times \text{max_depth}$

The learned weights (`derivation_layer`, `rule_head`) are constant regardless of `max_depth`. Only the depth embedding table grows with depth (256 bytes per level). This makes deep unrolling essentially free in parameter count.

Why Recursive?

The same grammar rules apply at every level of syntactic embedding — “the cat that the dog chased” uses NP expansion recursively. Weight sharing captures this: the layer learns a *single* rule-prediction function conditioned on derivation context (the hidden state `h`) and position (the depth embedding).

The depth embedding allows depth-specific behavior without depth-specific weights. Depth 0 can learn “always predict $S \rightarrow NP VP$ ” while deeper levels learn context-dependent rule selection — all through the same weight matrix receiving different input.

Differentiable Rule Selection

Rule selection uses **Gumbel-softmax** with temperature `tau`:

- **Training:** `F.gumbel_softmax(logits, tau=tau, hard=False)` produces soft one-hot vectors. Gradients flow through the reparametrization trick.
- **Eval:** `F.softmax(logits, dim=-1)` gives a deterministic distribution; `argmax` selects discrete rule IDs.
- **Temperature annealing:** `set_tau(tau)` controls the sharpness. Start at `tau=1.0` (soft), anneal toward `tau=0.1` (near-discrete) over training.

SyntacticSpace

Role

Generates a binary derivation tree (deep structure) from the set of active symbols produced by `SymbolicSpace`, using the learned `SyntacticLayer`.

Forward Path

1. **Extract activation.** Read the activation vector from the input subspace. If no activation exists (all symbols active), create a full-ones vector.
2. **Predict rules.** Pass activation to `SyntacticLayer.forward()`, which unrolls the recursive derivation stack and returns rule distributions at each depth plus assembled word tuples.
3. **Store derivation.** Word tuples are set on the output subspace. Vectors and activation pass through unchanged.

Reverse Path

1. **Decode derivation.** `SyntacticLayer.reverse()` walks the word list: every (`batch`, `vector`, `rule`) entry marks that position as active. This deterministically recovers the activation vector from the derivation alone.

2. **Store result.** Set the recovered activation on the output subspace (without recomputing from vector norms). Clear the word list.

Key Properties

- The derivation is a pre-order list of word tuples.
 - Reverse is **deterministic**: given the same word list, the same activation vector is always recovered.
 - The round-trip **forward** $\rightarrow$ (**delete activation**) $\rightarrow$ **reverse** preserves the original active positions exactly.
 - Default `max_depth = nSymbols - 1` (the maximum tree size for N leaves). Since the layer is recursive (shared weights), the only per-depth cost is one depth embedding vector — the parameter count is independent of depth.
-

Open Question: Differentiating the Rules Themselves

The grammar rules are currently labels — AND, OR, PART, etc. do not define distinct operations on their children's representations. The SyntacticLayer learns *which* rule to predict at each depth, but not *what* each rule does.

For the rules to be **meaningful**, each must define a distinct computation that transforms child representations into a parent representation:

- **Rule-specific layers.** Each rule maps to a learned transformation — e.g., AND might use a SigmaLayer (additive), UNION a max operation, etc. The `Grammar.perceptual()` and `Grammar.conceptual()` groupings suggest a natural partition: perceptual rules use PiLayer-style multiplicative operations, conceptual rules use SigmaLayer-style additive operations.
- **Shared layer with rule embeddings.** A single composition function conditioned on a learned rule embedding vector. Fewer parameters but less expressive per-rule behavior.

This remains the primary open design question for the language system.

Logic.md

Overview

This document defines a two-layer logic system:

1. **Subsymbolic (vector / field level)**
 2. **Symbolic (scalar level in [-1,1])**
-

1. Subsymbolic Layer

Objects:

- Vector sets: (B, N, D)
- Interpreted as RBF / luminosity fields

Operators

- **Union:** Combine sets
`union(A, B) = concat(A, B)`
- **Intersection:** Co-supported regions (RBF product / merge)
- **Negation (affirming):**

$$\text{neg}(x) = -x$$

Antipodal opposition on hypersphere

- **Non (non-affirming negation):**

$$\text{non}(x) = \alpha x, \quad \alpha \in [0, 1]$$

Contraction toward zero (withdrawal of assertion)

- **Parthood:** Fuzzy max-coverage:

$$P(A \rightarrow B) \in [-1, 1]$$

2. Symbolization

Map vectors $\rightarrow$ scalar truth strength

For $X \in (B, N, D)$:

$$s(X) = 2 \cdot \text{mean}(\|x_i\|) - 1$$

Range: $[-1, 1]$

Interpretation:

- $+1 \rightarrow$ strong presence
 - $0 \rightarrow$ neutral
 - $-1 \rightarrow$ absence
-

3. Symbolic Layer (Scalars in $[-1,1]$)

Let $a, b \in [-1, 1]$.

Negation (affirming)

$$\text{neg}(a) = -a$$

Non (non-affirming)

$$\text{non}(a) = \alpha a$$

Union

$$a \cup b = \max(a, b)$$

Intersection

$$a \cap b = \min(a, b)$$

Parthood (order relation)

$$\text{part}(a, b) = \text{clamp}(b - a, -1, 1)$$

4. Interpretation

Operator	Meaning
neg	oppositional negation
non	withdrawal / neutrality
union	strongest affirmation
intersection	shared commitment
part	signed containment

5. Key Insight

- Subsymbolic layer = geometry
- Symbolic layer = order + polarity
- Symbolization = norm projection

This cleanly separates:

- representation (vectors)
- logic (scalars)

Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel implements a novel approach to neural network optimization that decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two fundamentally different algorithms operate in tandem:

Component	Algorithm	What it does
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration level

The key insight: α **is not trained by gradient descent**. It is a *sensor* that reads the gradient energy flowing through the temperature parameter and converts it into an exploration policy.

1. The Ergodic Weights

In this model, weights are not treated as fixed constants. Each layer instead uses an effective weight matrix sampled from a locally specified ergodic distribution, with the learned tensor and the stochastic tensor defining the current mixture:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

So the layer never acts on a bare, timeless weight matrix; it acts on the current sample W_{eff} , while W stores the learned structure that increasingly shapes that local distribution as α rises. This perspective is adjacent to the broader literature on stochastic sampling in learning, including Monte Carlo methods, stochastic approximation, and noise-injected views of neural network optimization.

We define two derived quantities from the single scalar α :

$$\text{bias} = \alpha, \quad \text{var} = 1 - \alpha$$

This is a **direct encoding of the bias-variance tradeoff**:

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. The network is random noise.
- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. The network uses only learned weights.
- $0 < \alpha < 1$: Interpolation between learned structure and stochastic exploration.

Training begins at $\alpha = 0$ (full exploration) and the system autonomously transitions toward exploitation as the loss landscape flattens.

2. Why Standard Adam Fails on α

The temperature parameter $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is **re-sampled every forward pass**, this gradient is **zero-mean**:

$$\mathbb{E} \left[\frac{\partial \mathcal{L}}{\partial T} \right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \underbrace{\mathbb{E}[\varepsilon_{ij}]}_{=0} = 0$$

Standard Adam maintains a first moment (running mean) and a second moment (running variance), then computes the update as $m/\sqrt{v}$. For a zero-mean gradient:

$$m_t \approx 0 \quad \Rightarrow \quad \frac{m_t}{\sqrt{v_t}} \approx \frac{0}{\sqrt{v_t}} = 0$$

Adam produces no update. The first moment kills the signal. This is correct behavior for Adam — it correctly identifies that there is no consistent gradient direction — but it means Adam cannot be used to tune α .

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator

We discard the first moment entirely and use only the second moment, but as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ is the temperature gradient at step t , and $\beta = 0.999$ is the EMA decay rate.

3.2 Bias Correction

Identical to Adam's bias correction. Since $v_0 = 0$, early estimates are biased toward zero:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures

The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \text{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

This is the **squared Frobenius norm** of the loss gradient with respect to the effective weights — a scalar measure of how much the loss surface cares about weight perturbations. We call this the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss is sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit learned weights (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a tunable sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \rightarrow 0 \implies \alpha_t \rightarrow 1$ (exploit when gradients vanish)
- $\sqrt{\hat{v}_t} \rightarrow \infty \implies \alpha_t \rightarrow 0$ (explore when gradients are large)
- $\alpha_t \in (0, 1)$ always — the sigmoid-like shape prevents saturation
- τ controls the transition speed: large τ means more exploration at equivalent gradient energy

After computing α_t , the bias and variance are derived:

$$\text{bias}_t = \alpha_t, \quad \text{var}_t = 1 - \alpha_t$$

3.5 Layer-Local Certainty (Per-Neuron Sigma)

Each ergodic layer maintains per-neuron **sigma** — the running variance of its gradient energy — tracked via Welford’s algorithm in `observe_sigma()`. Sigma drives per-neuron bias and var:

$$\text{var}_j = \frac{\sigma_j}{\sigma_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

Low sigma (consistent gradient, found a minimum) yields high bias, low var. High sigma (unstable gradient) yields low bias, high var.

The `set_sigma(sigma)` method controls the sensitivity knob κ :

- **sigma=1**: encourage exploration (low κ , responsive to gradient variance)
- **sigma=0**: suppress exploration (high κ , var ≈ 0)

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Gradient Energy Sensor (for α)
Type	Optimizer (gradient descent)	Sensor (measurement)
First moment m_t	Yes — tracks gradient direction	No — direction is zero-mean, useless
Second moment v_t	Yes — normalizes step size	Yes — is the output
Role of $\sqrt{v_t}$	Denominator (adaptive learning rate)	Numerator (gradient energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m / \sqrt{v}$	$\alpha \leftarrow 1 / (1 + \tau \sqrt{v})$
Descent?	Yes — follows gradient downhill	No — maps energy to a policy
Zero-mean safe?	No — produces $0 / \sqrt{v} = 0$	Yes — by design

5. Derivation: Why Zero-Mean Implies Drop m_t

Let $g_t = \nabla_T \mathcal{L}$ be the temperature gradient. Since noise ε is i.i.d. each step:

$$g_t = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}^{(t)}$$

The first moment EMA is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Taking expectations:

$$\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}] + (1 - \beta_1) \underbrace{\mathbb{E}[g_t]}_{=0} = \beta_1 \mathbb{E}[m_{t-1}]$$

By induction: $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$. The first moment converges exponentially to zero. It carries **no information** about the loss landscape — only noise from finite sampling. Keeping it would add variance to the estimator without adding signal.

The second moment, however, converges to a meaningful quantity:

$$\mathbb{E}[v_t] \rightarrow \mathbb{E}[g_t^2] = \left\| \nabla_{W_{\text{eff}}} \mathcal{L} \right\|_F^2$$

This is exactly the gradient energy — the quantity we need.

6. Dropout via Bias

The **bias** and **var** parameters support **dropout regularization**:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1 - p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α . This is analogous to standard dropout but operates on the bias-variance tradeoff rather than individual activations. The temperature $T = 1 - \alpha$ remains unchanged so that the noise contribution is consistent.

Seen this way, dropout is just a schedule on bias. Ordinary Bernoulli dropout is the binary case in which the bias is either kept at α_t or forced to 0 for that step, while annealed dropout corresponds to increasing

the expected bias over training so the layer spends less time in pure-noise mode and more time exploiting learned structure. More generally, nonzero temperature bakes regularization directly into the model because every forward pass perturbs the effective weights, discouraging brittle co-adaptation and rewarding structure that survives stochastic variation.

7. The `global_temp` Sensitivity Knob

The parameter τ (`global_temp`) scales how responsive α is to gradient energy:

$$\alpha = \frac{1}{1 + \tau \cdot \sqrt{\bar{v}}}$$

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation, no exploration
$\tau \ll 1$	Aggressive exploitation — quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on a natural scale
$\tau \gg 1$	Conservative — maintains high exploration even with moderate gradients

In practice, τ can itself be scheduled (e.g., annealed from high to low) to implement a coarse exploration-to-exploitation curriculum.

8. Initialization and Bootstrap

Problem: If we initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and the temperature gradient $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero gradient energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution: Initialize at $\alpha = 0$ (pure exploration).

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- The temperature gradient is nonzero: $g_t = \sum_{ij} (\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- The sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, gradient energy decreases and α rises naturally

In this architecture, zero-initializing W is not only safe but often cleaner than random initialization. In an ordinary network, zero initialization is bad because it fails to break symmetry, but here symmetry is already broken by the sampled noise ε in W_{eff} . Since training begins at $\alpha = 0$, the learned weights do not contribute to the forward pass anyway, so random weight initialization only injects arbitrary structure into a phase that is meant to be pure exploration. Starting from $W = 0$ makes that exploratory regime honest and lets useful structure enter only when the bias toward learned weights increases.

9. Layer Architecture

All ergodic layers follow the effective-weight pattern. When `ergodic=True`, noise buffers are registered and weights are zero-initialized (noise provides initial variation). When `ergodic=False`, weights are randomly initialized and no noise buffers exist.

ErgodicLayer Base Class

ErgodicLayer (Model.py) provides per-neuron `bias` and `var` buffers driven by gradient variance tracking (`observe_sigma()` / `sigma_to_ergodic()`). Subclasses that contain child ErgodicLayers must forward `set_sigma()` and `paramUpdate()` to their children.

InvertibleLinearLayer (SVD decomposition)

$$W = U\Sigma V^\top$$

Factored into three ErgodicLayer sub-layers:

- **InvertibleRotationLayer** (U, V): Householder+diagonal+Householder. $R = H(v_1) D_{\{\text{sign}\}} H(v_2)$ where $H(\hat{v}) = I - 2\hat{v}\hat{v}^\top$. Applied directly to rows of x in $O(B \cdot N)$ via rank-1 updates.
- **InvertibleDiagonalLayer** (Σ): Element-wise multiply by singular values λ_k .

When `naive=False` (default), the fast $O(N)$ path applies sub-layers directly to x rows — each sub-layer handles its own ergodic mixing internally. When `naive=True`, W is materialized and mixed at the InvertibleLinearLayer level:

$$W_{\text{eff}} = \text{bias} \cdot W + \text{var} \cdot \varepsilon$$

InvertibleRotationLayer (Householder)

Parameterized by two direction vectors v_1, v_2 and a sign vector d :

$$\begin{aligned} v_k^{\text{eff}} &= \text{normalize}(\text{bias} \cdot v_k + \text{var} \cdot \varepsilon_k) \\ d^{\text{eff}} &= \text{sign}(\text{bias} \cdot d + \text{var} \cdot \varepsilon_d) \end{aligned}$$

Because `normalize` and `sign` always produce valid Householder/diagonal operators, the rotation remains orthogonal at all values of `bias/var`.

InvertibleDiagonalLayer

Parameterized by singular values λ_k (initialized to 1.0 in both ergodic and non-ergodic modes since zero singular values would kill the signal):

$$\lambda_k^{\text{eff}} = \text{bias} \cdot \lambda_k + \text{var} \cdot \varepsilon_k$$

10. Training Loop Integration

```
# 1. Standard Adam optimizer for W (excludes temperature)
optimizer = torch.optim.Adam(model.getParameters(), lr=lr)

# 2. Forward pass computes W_eff = bias*W + var*noise
loss = model(x, y)

# 3. Backward pass: Adam gets dL/dW, temperature gets dL/dT
loss.backward()

# 4. Adam updates W
optimizer.step()
```

```
# 5. The sensor updates alpha via paramUpdate() on each ErgodicLayer
#   set_sigma() propagates to child sub-layers
model.paramUpdate()
```

11. Summary of the Full Algorithm

Initialize:

$$\alpha_0 = 0, \quad v_0 = 0, \quad t = 0, \quad \beta = 0.999$$

Each training step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$
2. Compute $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t) \varepsilon$
3. Forward pass: $\hat{y} = f(x; W_{\text{eff}})$
4. Compute loss $\mathcal{L}(\hat{y}, y)$
5. Backward pass: compute $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$
6. **Adam** updates W : $W \leftarrow W - \eta \cdot \hat{m}_t / \sqrt{\hat{v}_t^{(W)}}$
7. **Sensor** updates α via per-neuron sigma tracking:

$$v_t \leftarrow \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

$$\text{var}_j = \frac{\hat{v}_j}{\hat{v}_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

8. Zero temperature gradient: $g_t \leftarrow 0$
-

Factor-Level Noise Injection

Previous Approach: Matrix-Level Blending

The original ergodic formulation applied noise at the level of the full weight matrix:

$$W_{\text{eff}} = b \cdot W + t \cdot N, \quad N \sim \mathcal{N}(0, I)$$

This worked well for non-invertible layers, but breaks down when exact invertibility is required. The approximate inverse

$$W_{\text{eff}}^{-1} \approx b \cdot W^{-1} + t \cdot N^{-1}$$

has reconstruction error proportional to the cross-terms:

$$W_{\text{eff}} \cdot W_{\text{eff}}^{-1} \approx (b^2 + t^2)I + bt(WN^{-1} + NW^{-1}) \neq I$$

The error is small when t is small, but grows with temperature — exactly the regime where exploration is most active.

New Approach: Factor-Level Injection

For `InvertibleLinearLayer`, noise is injected directly into the raw parameters of each LDU factor before the triangular structure is extracted:

```
L_eff = I + strict_lower(raw_L + t · noise_raw_L)
U_eff = I + strict_upper(raw_U + t · noise_raw_U)
d_eff = b · d_effective + t · noise_d
W_eff = L_eff @ D_eff_embed @ U_eff
```

Because W_{eff} is in LDU form regardless of the noise level, its exact inverse is always available via the same triangular solves:

$$W_{\text{eff}}^{-1} = U_{\text{eff}}^{-1} @ D_{\text{eff}}^{-1} @ L_{\text{eff}}^{-1}$$

No approximation is introduced at any temperature.

stable=True and the d Backbone

The `stable=True` flag controls whether `_d_effective()` clamps the deterministic diagonal backbone before the ergodic blend. This is the only place the stability constraint lives:

```
d_clamped_i = sign(d_i) * clamp(|d_i|, eps, 1)
d_eff = b · d_clamped + t · noise_d
```

The noise factor `noise_d` is sampled with magnitude in $[\text{eps}, 1]$ and random sign, so it is itself always invertible. Combined, `d_eff` stays bounded away from zero at all temperatures without any additional clamp on the blended result.

SigmaLayer and PiLayer Integration

`SigmaLayer(invertible=True)` and `PiLayer(invertible=True)` use `InvertibleLinearLayer(ergodic=True)` internally for the linear component of their transformations. The `bias` and `var` values computed by the parent `ErgodicLayer` are forwarded as `bias=` and `temp=` arguments to `InvertibleLinearLayer.forward()` and `InvertibleLinearLayer.reverse()`, so the same gradient-energy sensor that controls the outer layer also governs the factor-level noise injection in the inner linear primitive.

Training

Overview

BasicModel training has two phases: **embedding pretraining** and **network training**. Embedding pretraining builds word vectors from a large corpus. Network training uses those vectors as the input representation and learns to predict and reconstruct sentences.

The two phases can overlap: the `<trainEmbedding>` config controls whether and how embeddings continue to evolve during network training.

Phase 1: Embedding Pretraining (make train / embed.py train)

Produces a static embedding artifact (e.g. `BasicModel.kv`) containing word vectors. The output path is read from `<embeddingPath>` in the XML config.

Pipeline

FineWeb-EDU parquet shards

→ Pass 1: stream documents, lex + parse, count words, build vocabulary

- Pass 2: stream documents again, train SBOW per sentence, discard examples
- Save WordVectors to sentence.pt

SBOW (Sentence Bag of Words)

SBOW is a sentence-level variant of CBOW. Given a sentence of N words with embedding vectors and a linear output matrix with bias:

Leave-one-out centroid for word i :

$$c_i = \frac{1}{N-1} \sum_{j \neq i} v_j$$

Logits (unnormalised scores for every word in vocabulary V):

$$z_i = W_o c_i + b_o$$

Softmax probability that centroid predicts word i :

$$P(w_i | c_i) = \frac{\exp(z_{i,w_i})}{\sum_{k=1}^V \exp(z_{i,k})}$$

Loss (cross-entropy, averaged over all N sentence positions):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | c_i)$$

The gradient of the loss with respect to embedding v_i has two terms:

- **Attractive** (from position i where w_i is the target): pulls v_i toward directions that increase its probability.
- **Repulsive** (from positions $j \neq i$ where v_j is part of the centroid): pushes v_i away from centroids whose targets are other words.

The softmax partition function ensures that all V words compete: any word k too close to a centroid it doesn't belong to increases the denominator and is pushed away with force proportional to its softmax probability. This adaptive repulsion distributes vectors across the embedding space.

This gives N positive and $N \times (V-1)$ repulsive updates per sentence. Every word in the sentence is a positive example (unlike CBOW which picks one target).

```
vecs = embedding(idx)           # [N, dim]
total = vecs.sum(dim=0)         # [dim]
centroids = (total - vecs) / (N - 1) # [N, dim] leave-one-out
logits = linear(centroids)      # [N, vocab] full softmax
loss = cross_entropy(logits, idx)
```

CBOW (Continuous Bag of Words)

CBOW predicts a single target word from its context. Given a sentence of N words, for target word w_i with context $C_i = \{w_j : j \neq i\}$:

Context mean (with padding for variable-length contexts):

$$\bar{c}_i = \frac{1}{|C_i|} \sum_{j \in C_i} v_j$$

The loss is the same negative-sampling objective as SBOW but applied per-word to the padded context mean rather than the leave-one-out centroid:

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma(\bar{c}_i^\top v_{w_i}) - \sum_{k=1}^K \log \sigma(-\bar{c}_i^\top v_{w_k^-})$$

where K is the number of negative samples and w_k^- are randomly sampled words.

Two-Pass Architecture

The trainer uses two passes over the data to avoid dynamic model resizing:

- **Pass 1 (vocabulary):** Stream all documents, count every word via `lex + parse`. Words meeting `min_count` are promoted to the trainable vocabulary. The model (embedding + linear head) is allocated once at the final vocabulary size.
- **Pass 2 (training):** Stream documents again. For each sentence, run one SBOW step and immediately discard the examples. No examples are accumulated in memory. Multiple epochs re-stream the data.

Configuration (Makefile variables)

Variable	Default	Description
BASIC_SHARDS	1	Number of FineWeb-EDU parquet shards
BASIC_MAX_DOCS	10000	Maximum documents to process
BASIC_VEC_SIZE	100	Embedding dimensionality
BASIC_EPOCHS	10	Training epochs
BASIC_MIN_COUNT	10	Minimum word count for vocabulary inclusion
BASIC_BATCH_SIZE	8	Batch size (unused by SBOW; per-sentence training)

Memory Considerations

The dominant memory cost is the linear output head: `vocab_size × vector_size` parameters plus optimizer state. With 200K vocabulary and 100 dimensions:

- Embedding: $200\text{K} \times 100 \times 4 \text{ bytes} = \sim 80\text{MB}$
- Linear head: $100 \times 200\text{K} \times 4 \text{ bytes} = \sim 80\text{MB}$
- Adam optimizer: $2 \times \text{momentum buffers} = \sim 320\text{MB total}$

The full softmax computes $(N, \text{vocab_size})$ logits per sentence. For large vocabularies this can be expensive. Training runs on CPU by default (`BASICMODEL_DEVICE=cpu`) to avoid MPS/GPU memory limits.

Phase 2: Network Training (`BasicModel.py`)

The network learns to predict and reconstruct sentences using the pretrained embeddings as its input representation.

Masked Prediction Modes

The `maskedPrediction` config controls the prediction objective:

Mode	Input Masking	Truncation	Target	Description
NONE	None	No	Dataset labels	Standard supervised
MLM	Zero word i	No	Embedding of word i	Bidirectional (BERT-style)
ARLM	Zero word i	Yes (j > i zeroed)	Embedding of word i	Autoregressive (GPT-style)
ARUS	Same as ARLM	Yes	Zero vector	Unsupervised (loss suppressed)
RARLM	Zero word i	Reverse (j < pos zeroed)	Embedding of word i	Right-to-left autoregressive

<trainEmbedding>: Embedding Update Mode

The `trainEmbedding` config controls whether and how embeddings evolve during network training. When embedding updates are enabled (CBOW, SBOW, or BOTH), this implements an EM-like alternation:

- **E-step (network):** Forward pass + masked prediction + backward + optimizer step. The network learns to predict/reconstruct with the current embedding space.
- **M-step (CBOW/SBOW):** After each network step, run one embedding update on the same sentence. The embedding vectors move to better capture co-occurrence.

Value	Embedding method	Model layers	Description
NONE	Frozen	Trained	Embeddings frozen; only model layers train
CBOW	CBOW (padded context)	Trained	CBOW reshapes embeddings via own optimizer; model layers train
SBOW	SBOW (centroid)	Trained	Faster variant: predict word from leave-one-out centroid
BACKPROP	Backprop only	Trained	Codebook trained purely via model loss gradients; no SBOW/CBOW
BOTH	SBOW post-batch	Trained	Two optimizers: SBOW updates embeddings, Adam updates model
JOINT	Single backward	Trained	Single optimizer: combined model + SBOW loss

Gradient Flow Through Codebook

The `<trainEmbedding>` mode determines whether the codebook weight tensor is **detached** during forward/reverse passes and whether it appears in the main optimizer's parameter list.

<code>trainEmbedding</code>	<code>detach()</code> in forward/reverse	In main optimizer	Embedding method
NONE	Yes — codebook is a frozen lookup	No	Frozen
CBOW	Yes — codebook is a frozen lookup	No	CBOW (own optimizer)
SBOW	Yes — codebook is a frozen lookup	No	SBOW (own optimizer)
BACKPROP	No — gradients flow through codebook	Yes	Model loss only
BOTH	No — gradients flow through codebook	Yes	SBOW + backprop
JOINT	No — gradients flow through codebook	Yes	Single combined loss

NONE is the recommended mode for constrained hardware (e.g. Apple MPS with <16GB). The codebook from Phase 1 is frozen, all memory and compute go to the model layers.

CBOW/SBOW maintains clean EM separation: CBOW/SBOW reshapes the embedding space using its own optimizer. No gradients flow from the model through the codebook.

BACKPROP is the simplest trainable mode. The codebook participates in the main optimizer and receives gradients from the model's output and reconstruction losses, but no embedding-specific loss (SBOW/CBOW) is applied. The embedding space is shaped entirely by the task. Best for small vocabularies or when the pretrained embedding geometry should adapt freely to the downstream objective.

BOTH risks gradient interference. The network’s reconstruction loss pulls embeddings toward being maximally distinguishable (spreading apart), while SBOW pulls co-occurring words together. These forces can conflict because they use separate optimizers with separate momentum buffers.

JOINT: Single Combined Loss

JOINT mode avoids the EM alternation of BOTH by computing a single combined loss before one backward pass:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$$

where $\mathcal{L}_{\text{model}}$ is the standard output loss (plus reconstruction loss if `reversible=true`), and λ is the `trainEmbeddingRatio` hyperparameter (default 0.1).

The SBOW loss for the current sentence is computed using the same negative-sampling objective as Phase 1:

$$\mathcal{L}_{\text{SBOW}} = -\frac{1}{N} \sum_{i=1}^N \left[\log \sigma(c_i^\top v_{w_i}) + \sum_{k=1}^K \log \sigma(-c_i^\top v_{w_k^-}) \right]$$

where c_i is the leave-one-out centroid, v_{w_i} is the target word’s embedding, and w_k^- are negative samples drawn uniformly from the vocabulary.

Advantages over BOTH:

- **One optimizer, one momentum buffer.** Adam sees the full gradient landscape from both objectives simultaneously, avoiding the oscillation risk of two independent optimizers pulling embeddings in different directions.
- **Gradient coherence.** The model’s MSE loss and the SBOW co-occurrence loss are balanced by λ before any parameter update, so the step direction reflects the intended trade-off.
- **Simpler implementation.** No post-batch embedding step; everything flows through `totalLoss.backward()`.

Trade-off: Both objectives share the same learning rate and Adam state. If the two loss surfaces have very different curvature, `trainEmbeddingRatio` may need tuning. The BOTH mode’s separate optimizers can use different learning rates.

Why MSE over Embeddings (not Cross-Entropy over Vocabulary)

The network’s output loss uses MSE against the target word’s embedding vector, not cross-entropy over vocabulary indices. This is an architectural advantage:

- A prediction near “cat” when the target is “kitten” incurs less loss than a prediction near “democracy” — the embedding space captures semantic similarity.
- Output dimensionality is the embedding dimension (~100) rather than the vocabulary size (~200K), reducing parameter count.

Training Loop

Each batch (one sentence in masked prediction mode):

1. Embed sentence, expand to N masked copies (one per word position)
2. Forward through network: `InputSpace` → `Percept` → `Concept` → `Symbol` → `Output`
3. Compute output loss (MSE against target word embeddings)
4. If reversible: reverse pass reconstructs input, compute reconstruction loss
5. Combined loss = $(1 - \text{recon_ratio}) \times \text{output_loss} + \text{recon_ratio} \times \text{recon_loss}$
6. If `train` is JOINT: add `trainEmbeddingRatio` × `sbow_loss` to the combined loss

7. Backprop + optimizer step (includes embedding params for BACKPROP, BOTH, JOINT; excludes for NONE, CBOW, SBOW)
8. If `train` is CBOW, SBOW, or BOTH: run embedding step on same sentence
9. (removed — [MASK] is just the zero vector, not a vocabulary entry)

SBOW vs CBOW

Property	CBOW	SBOW	Masked Prediction (Phase 2)
Targets per sentence	1 (pick one word)	N (every word)	N (one per masked position)
Context	Other N-1 words	Leave-one-out centroid of N-1	All unmasked words (+ zero vector)
Positive updates	1 per step	N per step	N per step
Repulsive force	vocab-1 implicit via softmax	$N \times (\text{vocab}-1)$ via softmax	Implicit via MSE in embedding
Signal density	Low (1 gradient per sentence)	High (N gradients per sentence)	High (N gradients per sentence)
Loss function	Cross-entropy over vocabulary	Cross-entropy over vocabulary	MSE over embedding vectors
Updates embeddings directly	Yes (own optimizer)	Yes (own optimizer)	Only for BACKPROP/BOTH
Used in (<code><trainEmbedding></code>)	CBOW	SBOW, BOTH, JOINT	BACKPROP, BOTH, JOINT

SBOW provides richer signal per sentence because every word acts as both context (for other words) and target (predicted from others). The full softmax over the vocabulary at each position ensures vectors distribute across the hypersphere: words too close to centroids they don't belong to are pushed away with force proportional to their proximity.

Three-File Architecture

Model behaviour is partitioned across three independently managed artifacts:

Artifact	Example	Contents	Updated by
XML config	<code>data/BasicModel.xml</code>	Architecture, hyperparameters, file paths	Hand-edited
Embedding	<code>data/BasicModel.kv</code>	Word vectors ($V \times d$ codebook matrix)	Phase 1: <code>embed.py</code>
Weights	<code>data/BasicModel.ckpt</code>	Model layer parameters (attention, Pi/Sigma weights)	Phase 2: <code>BasicModel.py</code>

The checkpoint explicitly excludes embedding parameters (`wv.vectors`). This separation enables independent iteration:

- Retrain embeddings without touching model weights (`--force-embeddings`)
- Retrain model with frozen codebook (`<trainEmbedding>NONE</trainEmbedding>`)
- Swap codebooks between models sharing the same architecture

Embedding Space Geometry

The full softmax provides a distributional guarantee that random negative sampling cannot match. For predicting word `w_i` from centroid `c_i`, the loss is:

$$\mathcal{L}_i = -\log \frac{\exp(z_{i,w_i})}{\sum_{j=1}^V \exp(z_{i,j})}$$

where the logits are:

$$z_i = W_o c_i + b_o$$

The partition function (denominator) sums over every word in the vocabulary. The gradient with respect to the logit for any non-target word $k \neq w_i$ is:

$$\frac{\partial \mathcal{L}_i}{\partial z_{i,k}} = P(k | c_i)$$

Words with high softmax probability — those too close to the centroid — receive the strongest repulsive gradient. Words already far away have near-zero probability and are left alone. This adaptive repulsion naturally spreads vectors across the hypersphere.

At equilibrium, no word can move closer to any centroid without increasing the loss. This is the key advantage over approaches like direct push/pull with random negatives, where most sampled negatives are already far away on a high-dimensional hypersphere and contribute little useful gradient.

Profiling

The `--profile` flag wraps Phase 2 in `cProfile`:

```
make train_micro_remote # add --profile via train.py
# Or directly:
python bin/train.py --model data/BasicModel.xml --profile --max-docs 500
```

Output: a `.prof` file in `output/profiles/` plus a top-30 summary printed to stdout. View interactively with `snakeviz`:

```
pip install snakeviz
snakeviz output/profiles/train_20260319_111441.prof
```

For live profiling of a running process, `py-spy` can attach by PID (requires root on macOS):

```
pip install py-spy
sudo py-spy record -o profile.svg --pid <PID> --duration 30
```

Alec Rogers

September 24, 2025

Abstract

Problem

Society has become highly dependent on machine minds, but we do not have a good theory of those minds. Although human and machine minds have considerable functional overlap, their design goals are considerably different, and treating a machine mind as a human mind can have significant negative consequences. Therefore, we should have a different “theory of mind” when we interact with machine minds, and our theory should be informed by how those minds are designed and trained.

Background

To develop this theory of mind, it is helpful to anthropomorphize machines because we naturally understand them in relation to human minds. Therefore, this essay focusses on three main questions: what are machine minds, what do machine minds feel, and what do machine minds know.

While these changes do not allow us to adopt a human theory of mind, they do permit of a theory of mind that is significantly less broken and inhumane.

Solution

As a result of this analysis, we also propose three specific measures to make a machine mind more humanistic or natural: *weight ergodicity*, *network invertibility*, and *output certainty*. *Weight ergodicity* refers to the fact that weights are samples from a random (ergodic) process, rather than fixed constants. *Network invertibility* refers to the fact that the network architecture is bidirectional and partially invertible. *Output certainty* refers to the fact that a network should know that it does not know, as opposed to merely knowing the probability of one answer as opposed to another. To illustrate these principles, the performance of several models implementing these paradigms is compared to a traditional model. All models are configured with a 20-neuron, single hidden layer and tested on the MNIST dataset.

Network invertibility is important with respect to developing a meaningful semantic embedding: the symbols of an LLM should correspond to something and we should be able to know to what they correspond, as opposed to being merely abstract tokens inside of Searl's Chinese translation engine.

Background

The scope of this paper is Large Language Models, or reinforcement learning where the inputs and outputs are known and the weights of the network are unknown (i.e. the transformer architecture).

Part I: What Machine Minds Are — Weight Ergodicity

Weights as Ergodic Samples

Measurement theory, when applied to a neural network, views the weights not as fixed constants to be optimized but as **samples from an ergodic distribution**. The weight matrix W at any moment is one realization of a stochastic process whose statistics reflect the boundary conditions of the learning problem:

$$W = \mu M + \sigma V$$

where μ and σ are the bias and variance parameters that determine the contribution of the random process. The bias and variance parameterize a noisy signal where M has a norm of 1 and variance of zero and an unbiased variance V with unit variance. Note that the matrix M is analogous to the weight matrix in a typical neural network while V is sampled from a standard normal distribution.

This perspective comes from **Lagrangian Field-theoretic Gradient Control** (LFGC), which treats the weight landscape as a field where the boundary conditions — input data, output targets, and architectural constraints — shape the distribution from which weights are drawn. Rather than searching for a single optimal point, the network explores the manifold of solutions consistent with its boundary conditions. The ergodic hypothesis guarantees that time averages (over training steps) equal ensemble averages (over weight configurations), so the training trajectory visits representative regions of the solution manifold.

See Rogers, A. (2026), *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

Temperature and Annealing

For simplicity, the mean and variance of all weights are characterized here by a single *temperature* parameter for each layer that is updated via the ADAM algorithm. While it may be desirable to have a temperature associated with each input weight, this is not necessary to compare to a normal network in order to measure performance.

Another simplification in this context is to use the temperature parameter to determine both the bias and the variance. Given the temperature *temp*, the μ and σ parameters are calculated as follows:

$$\mu = 1 - temp$$

$$\sigma = temp$$

In this experiment, the temperature of the ergodic weight process begins at 1 and decreases to 0 using the ADAM algorithm.

Advantages of Ergodic Weights

There are three significant advantages to using ergodic weights:

1. **No learning rate required.** Biases find different locations in weight space as a result of the randomness and converge as the temperature decreases. The adaptive learning rate schedule is reinterpreted as a process of simulated annealing.
2. **Zero initialization.** Weights can be initialized to zero, since they find different locations in weight space over time. The annealing process helps the model bounce out of local minima and saddle points more effectively than stepping opposite to the gradient, which is prone to deterministic and oscillatory behavior.
3. **Theoretical convergence guarantee.** The literature on simulated annealing provides a theoretical guarantee of convergence to the global optimum if run for an infinite number of iterations.

Note that the standard *dropout* algorithm may be interpreted as a per-neuron variance in the bias parameter. Thus, while dropout explicitly sets neurons to zero with some frequency, the same type of regularization will occur naturally as a result of the difference over time of the weight biases.

Part II: What Machine Minds Feel — Network Invertibility

Bidirectional Perception

The egocentric point of view tends to see perception as a passive process, while physics suggests that it is bidirectional. That bidirectionality within a neural network can be approximated via the invertibility of its weight multiplications and activation functions.

Symbols, Worldlines, and Karmic Inertia

The symbols or tokens that an LLM processes are not arbitrary labels — they have meanings given to them by **worldlines**. A worldline is the trajectory of a concept through training: the accumulated history of contexts in which a token has appeared, the gradients that have shaped its embedding, and the network states it has participated in. The meaning of a symbol is the integral of its worldline.

Weight adaptation — or even inference — puts a **force** into the system. That force will adapt either the system’s inputs or its outputs, depending on which has less **karmic inertia**. Karmic inertia is the resistance of a representation to change: a well-established embedding with a long worldline (many training examples, consistent gradients) has high karmic inertia, while a novel or ambiguous token has low karmic inertia. The network’s bidirectional architecture means this force propagates in both directions:

- **Forward (encoding):** the input representation adapts to match the network’s learned categories — perception is shaped by expectation.
- **Reverse (decoding):** the network’s internal state adapts to reconstruct the input — expectation is grounded in perception.

The invertible architecture makes both directions explicit and trainable, rather than relying on separate encoder/decoder networks with independent weights.

Implementation

Even when the network is not invertible, bidirectionality provides an important role in backpropagation and network topologies such as autoencoders that perform a decoding step that is analogous to the reverse of the encoding step. In modern practice, the topological similarity of encoders and decoders does not typically extend to shared weights. Here, we briefly examine the possibility of simultaneously training the network in both the forward and reverse direction: the forward encoding step in which the outputs are predicted from the input, and the reverse decoding step in which the inputs are predicted from the (possibly predicted) outputs. For a related experiment in sharing the weight matrix between controller (policy) and critic (value) networks, see *Rogers et al 2001*.

In this experiment, the output is a one-hot encoded vector corresponding to the predicted MNIST digit, a classification task that is clearly not a bijective (invertible) function. To address this situation, the inputs are predicted from the outputs of the *penultimate* network layer; thus, the network is composed of a bijective front and a surjective back. We characterize these components respectively as the *recognizer* (or representer) and the *generalizer*.

Part III: What Machine Minds Know — Output Certainty

Certainty vs. Probability

The error function of a neural network corresponds to its desire. Modern machine minds use cross-entropy loss, so their output distribution *must* classify the output. This does not allow the network to express the certainty that the input belongs to a given class or not; rather, it expresses the probability that the input belongs to one class *as opposed to another*.

To remedy this situation, a variation of cross-entropy loss is used which blends standard cross-entropy loss with a product of cross-entropy loss and the norm of the prediction. An output value of zero is penalized by cross-entropy loss, but any incorrect prediction carries an additional penalty in proportion to its magnitude to penalize overconfidence.

Certainty-Weighted Loss

The formula for the certainty-weighted cross-entropy version of the loss function is:

$$error = -\sum_{c=1}^N t_c \log(p_c)$$

$$certainty = |\hat{y}|$$

$$surprise = (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error$$

cosine similarity

Amplitude certainty

$$error = (\hat{y} - y)^2$$

Knowing What You Do Not Know

The key insight is that a network should **know that it does not know**, as opposed to merely knowing the probability of one answer as opposed to another. Cross-entropy forces the network to distribute all of its probability mass across the known classes, even when the correct response is “I don’t know.” Certainty weighting allows the network to express low confidence (small output norm) independently of its class distribution, separating the questions “which class?” and “how sure?”

Methods

The software used to run these experiments is a combination of Python and PyTorch (the Python code is provided in the appendices).

The data is the MNIST dataset, chosen because it is simple and well-known. It consists of 60,000 training images and 10,000 test images, each 28x28 pixels in size, with a monochromatic bit depth of 256. Their target labels are provided as output. The data is preprocessed by removing its mean and variance, and is shuffled between trials.

For all paradigms, the input is a 28x28 element array and the output is a one-hot encoded representation of the target class. The architecture for all paradigms is a 20 hidden unit neural network. The batch size is 10, and each network is run for 9 epochs. To generate confidence intervals for model accuracy, each paradigm is run for 7 trials to generate error bars reflecting the standard deviation of the accuracy values.

The standard architecture that serves as a benchmark for the other paradigms uses a ReLU nonlinearity, and a learning rate of 0.01 that is updated via the ADAM method. Cross-entropy loss is used for the error function.

The base ergodic model uses weights initialized to zero and a single per-layer temperature parameter that governs the bias/variance trade-off of the ergodic weights. The temperature is updated with the ADAM method. Dropout is used for the middle layer, which means that both the bias and variance of the ergodic weights are set to zero with a frequency of 0.75.

Five models were evaluated: the base model and four models using the ergodic weight updating scheme. Of the models with ergodic weights, one model has simply ergodic weights, one has input normalization, one reversible model uses separate forward and backward layers, and one reversible model uses a single invertible weight matrix. For the invertible model, the inversion of the weight matrix is typically computationally expensive. Therefore, the weight matrix is represented in SVD form, as a rotation matrix followed by an eigenvalue matrix followed by another rotation matrix. In virtue of this sparsely-parameterized decomposition, the rotation matrices can be computed by Givens rotations, and the eigenvalue matrix stores only diagonal elements. Therefore, direct matrix inversion and its computational cost are avoided. That said, doing so requires a more computationally expensive forward pass and a somewhat circuitous back propagation step.

The weight update equations for the ergodic model do not use a learning rate. So compared to traditional gradient descent equations that look as follows for a given learning rate parameter α :

$$err = (y - \hat{y})^2$$

$$\Delta_W = \alpha(y - \hat{y})\nabla_F x$$

$$err = (y - \hat{y})^2$$

$$e = y - \hat{y}$$

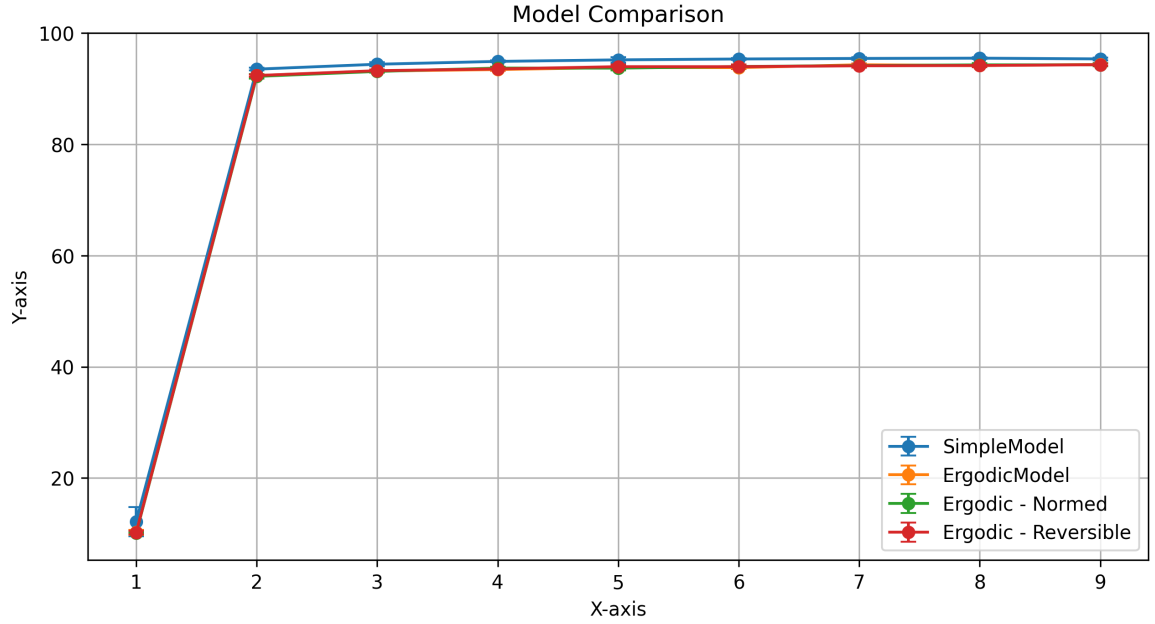
$$c = \alpha \hat{y}^2 + (1 - \alpha)$$

$$\Delta_W = (ce - \alpha \hat{y} e^2)x$$

This equation demonstrates that the network is simultaneously trying to minimize error and “incorrect certainty”.

Results

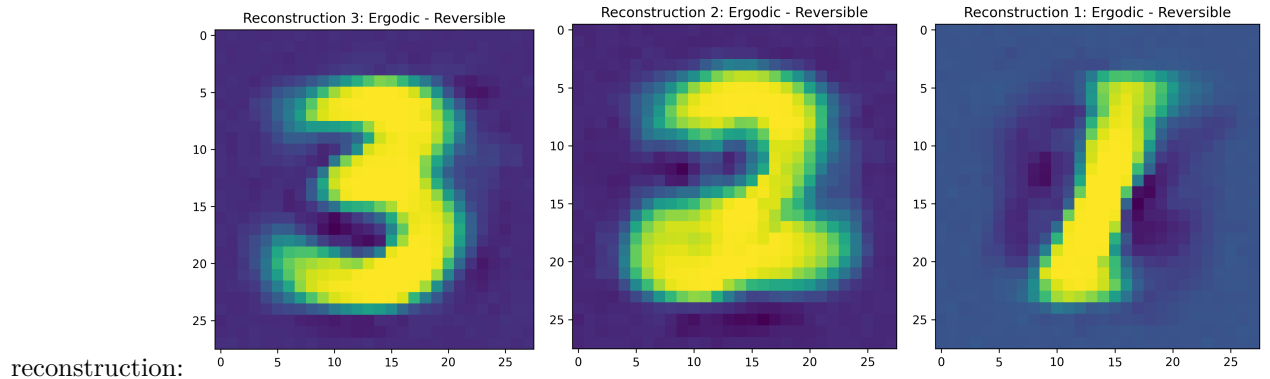
The most important result of this experiment is the model comparison with error bars, that demonstrates that the simple model performs slightly better than all proposed paradigms, and that all of the proposed paradigms



perform a
identically:

lmost

Finally, although there is no invertible network to compare it to, we show the reconstruction of the input by running the network in reverse, which uses a separate weight matrix and MSE loss for input



reconstruction:

Discussion

The benefits of the novel paradigms described in this paper are as follows:

Random weight initialization is unnecessary, which makes performance less subject to arbitrary initialization. This is visually demonstrated by the error bars in the Model Comparison figure, which show that only the simple model (with random weight variation) has significant variation in performance.

The learning rate is unnecessary, since its role is replaced with weight noise variance, just as the role of dropout is replaced with weight bias variance (which in this experiment is just the regular dropout schedule). Since dropout is folded into the weight adaptation schedule, an integrated algorithm that controls both weight bias and weight variance (and which uses separate parameters for each weight) is suggested for future experiments that follow this paradigm.

The error function encapsulates a measure of certainty in the output in addition to a measure of certainty between the output choices.

However, there were several experiments that my hardware was not fast enough to evaluate and a number of implementation issues that I did not have the time to address:

The Pearson correlation between the certainty and the accuracy for each digit is insignificant, which suggests that the error function did *not* induce the network to be less certain when it was less accurate. This probably points to a problem in the implementation of the error function: instead of using $\text{certainty} = |\hat{y}|$ to reduce the penalty for lack of certainty, we used $\text{certainty} = p_c$. This leads to prediction norms that are much higher than 1.0 (since softmax creates a unit-sum PDF at the output layer, while using the norm of the output vector to indicate certainty imposes a contradictory constraint). This might explain the lack of correlation between the certainty and accuracy, which theoretically should correlate quite strongly. Performing the experiment with corrected code (as by using a norm-weighted MSE with only one zero) should yield better correlation between certainty and accuracy for each target class.

Finally, **the use of the autograd algorithm within PyTorch led to weights being tuned directly**, which interferes with the simultaneous tuning of the temperature parameter in a way that is not explicitly captured in this paper. So while the addition of weight variance does make random weight initialization unnecessary, the intricacies of the interaction between the weight tuning and the temperature tuning is hidden behind the autograd mechanism.

Conclusion

Overall, I am optimistic about the strength of the philosophical grounding behind several of the paradigms introduced in this paper. However, due to several implementation issues, I do not feel that the hypotheses were well-tested. Further, even if they had been, the effectiveness of these neural network paradigms in both multilayer neural networks and with more challenging tasks is not clear.

References

- Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, Proceedings of the Joint 9th IFSA World Congress and 20th NAFIPS International Conference, IEEE
- Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished
- Rogers, A; 2026: *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302

XML Configuration Parameters

Model configuration is specified in XML files (e.g. `data/XOR_exact.xml`). Default values are loaded from `data/model.xml` and overridden by model-specific configs. The schema is defined in `data/model.xsd`.

<architecture>

Core model settings. Training and data parameters are in nested sub-elements <training> and <data> (see below).

Parameter	Type	Default	Description
modelType	string	"simple"	Model type: "simple" (feedforward), "lm" (language model with sequence processing).
ergodic	bool	false	Enable ergodic exploration mode. Layers use $W_{\text{eff}} = \text{bias} * W + \text{temp} * \text{noise}$.
certainty	bool	false	Enable per-neuron certainty tracking in ergodic layers. Allows individual neuron activation.
reshape	bool	false	Reshape input data for sequence processing. Required for modelType=lm.
conceptualOrder	int	1	Order of conceptual processing. Controls how many higher-order conceptual transformations.
symbolicOrder	int	1	Order of symbolic processing. Controls how many higher-order symbolic transformations.
processSymbols	bool	false	Enable additional symbolic processing steps.
maskedPrediction	string	"NONE"	Masked prediction mode: NONE, MLM, ARLM, ARUS, RARLM. See Training.md.
reconstruct	string	"NONE"	Controls the reverse (reconstruction) pass. "NONE" disables reconstruction entirely.
maxResponseLength	int	64	Maximum number of characters/tokens to generate during inference. Measured in characters.

<architecture><data>

Data loading and filtering settings.

Parameter	Type	Default	Description
dataset	string	"xor"	Dataset to load. Determines input/output data via <code>TheData.load()</code> . Common values are "xor", "word", "image", "audio", "video", "text", "image_text", "audio_text", "video_text".
minFrequency	float	0.0	Minimum word frequency ratio for vocabulary admission. Words below this threshold are ignored.
shardDir	string	—	Directory containing text shards for streaming datasets (e.g. "data/fineweb").
numShards	int	1	Number of shards to load from shardDir.
maxDocs	int	10000	Maximum number of documents to load per shard.
classificationMin	float	—	Minimum threshold for classification accuracy reporting.
classificationMax	float	—	Maximum threshold for classification accuracy reporting.

<architecture><training>

Training loop and I/O settings.

Parameter	Type	Default	Description
numTrials	int	1	Number of independent training runs.
numEpochs	int	3	Number of training epochs per trial.
batchSize	int	10	Mini-batch size for training.
learningRate	float	0.001	Learning rate for the Adam optimizer.
reconRatio	float	0.5	Weight of reconstruction loss in combined loss: $\text{total} = \text{recon} + \text{task}$.
trainEmbedding	string	"NONE"	Embedding update mode. Controls both the embedding training and inference.
trainEmbeddingRatio	float	0.1	Weight λ of embedding loss in JOINT mode: $\mathcal{L} = \mathcal{L}_{\text{model}} + \lambda \mathcal{L}_{\text{embed}}$.
weightsPath	string	"output/BasicModel.ckpt"	File path for saving/loading model weights checkpoint. File must exist for loading.
embeddingPath	string	—	File path for the word vector store (.kv extension, gensim format).
autoload	bool	true	Automatically load weights from weightsPath on model creation.
autosave	bool	false	Automatically save weights after training completes.
negSamples	int	64	Number of negative samples per positive example for CBO.

<trainEmbedding> — Embedding Update Modes

Value	Embedding method	Model layers	Gradients through codebook	Description
NONE	Frozen	Frozen	No	Inference only; no parameters updated
CBOW	CBOW (padded context)	Frozen	No	True CBOW: predict each word from lea
SBOW	SBOW (centroid)	Frozen	No	Faster variant: predict each word from le
ARLM	Frozen	Backprop	No	Train model layers only; codebook is fixe
BOTH	SBOW post-batch	Backprop	Yes	Two optimizers: SBOW updates embedd
JOINT	Single backward	Backprop	Yes	Single optimizer: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{codebook}}$

<trainEmbeddingRatio> — Embedding Loss Weight (JOINT mode)

Parameter	Type	Default	Description
trainEmbeddingRatio	float	0.1	Weight λ of the embedding co-occurrence loss in JOINT mode. Higher values bi

<InputSpace>

The entry point that lifts raw data into the model's internal representation.

Parameter	Type	Default	Description
nActive	int	<i>required</i>	Sequence length: maximum number of tokens per input. For XOR: 2 (two binary inputs)
nDim	int	1	Dimensionality of each input vector. Set on TheObjectEncoding via XML; not passed to
nVectors	int	= nActive	Codebook size (total vectors in the space). Defaults to nActive for InputSpace.
nWhere	int	0	Number of spatial/positional dimensions appended to each vector. When > 0, enables P
nWhen	int	0	Number of temporal dimensions appended to each vector. When > 0, enables Temporal
quantized	bool	false	Whether input values are quantized/discrete.
lexer	string	"word"	Tokenization mode for text inputs. Common values in the current configs are "word" an

Note: InputSpace's `inputShape` uses the data's native dimension (e.g. 784 for MNIST, `inputLength` for text), while `outputShape` uses `nDim` from TheObjectEncoding. The LiftingLayer bridges the two.

Layers: Contains a `LiftingLayer` that projects raw input into the model's working dimensionality.

<PerceptualSpace>

Transforms lifted input into perceptual features via Pi layers (multiplicative interactions).

Parameter	Type	Default	Description
nActive	int	<i>required</i>	Number of active perceptual feature vectors (sparse subset selected in forward pass). Sh
nDim	int	1	Dimensionality of each perceptual vector. Set on TheObjectEncoding; not passed to the
nVectors	int	0	Codebook size (total vectors in the space). When > 0, enables vector quantization with
invertible	bool	false	Use a single invertible Pi layer instead of separate forward/reverse layers. When true , o
hasAttention	bool	true	Enable attention mechanism in this space.
passThrough	bool	false	Skip perceptual processing entirely; pass input through unchanged.

Layers: Pi layers — multiplicative: $y_j = b_j * \prod_i (1 + W_{ji} * x_i)$. See Architecture.md.

Note: `InvertiblePiLayer` has been merged into `PiLayer(invertible=True)`; the standalone class is removed.

<ConceptualSpace>

Transforms perceptual features into abstract concepts via Sigma layers (additive/linear).

Parameter	Type	Default	Description
<code>nActive</code>	int	<i>required</i>	Number of active concept vectors (sparse subset). For XOR: 3.
<code>nDim</code>	int	1	Dimensionality of each concept vector. Set on <code>TheObjectEncoding</code> ; not passed to the Sigma layer.
<code>nVectors</code>	int	0	Codebook size (total vectors in the space).
<code>invertible</code>	bool	false	Use single invertible Sigma layer vs. separate forward/reverse layers (<code>sigma1/sigma2</code>).
<code>hasAttention</code>	bool	false	Enable attention in conceptual processing.
<code>hasNorm</code>	bool	false	Enable layer normalization in this space.

Layers: Sigma layers — additive: $y_j = \tanh(W x + b)$. See `Architecture.md`.

Note: `InvertibleSigmaLayer` has been merged into `SigmaLayer(invertible=True)`; the standalone class is removed.

<SymbolicSpace>

Discrete symbolic representation — the information bottleneck between perception and output. Maps concept activations to a one-hot encoding over a codebook of symbol prototypes. See `Language.md` for the full design.

Parameter	Type	Default	Description
<code>nActive</code>	int	<i>required</i>	Number of active symbols. When reconstruction symbols are enabled, <code>nOutputSymbols</code> must be <code>> 0</code> .
<code>nDim</code>	int	1	Dimensionality of each symbol (typically 1 — symbols are scalar activations).
<code>nVectors</code>	int	<code>= nActive</code>	Codebook size. When <code>quantized=true</code> , equals the number of symbol prototypes.
<code>passThrough</code>	bool	false	Pass concepts through as symbols unchanged. Typically true for simple models.
<code>quantized</code>	bool	false	Enable codebook quantization. When true , the forward path produces a one-hot activation over the codebook.

Layers: `InvertibleLinearLayer(nConcepts, nSymbols)` maps between concept and symbol activation spaces. Codebook provides dense vectors when quantized.

<SyntacticSpace>

Generates binary derivation trees (deep structure) from active symbols using a Chomsky Normal Form grammar. See `Language.md` for the grammar and open implementation questions.

Parameter	Type	Default	Description
<code>nActive</code>	int	16	Number of active word vectors.
<code>nDim</code>	int	1	Dimensionality of each word vector.
<code>nVectors</code>	int	<code>= nActive</code>	Codebook size (total vectors in the space).

Layers: No trainable parameters currently. Rule selection is random; derivation is stored as word tuples (`batch`, `vector`, `rule`) on the output subspace.

<OutputSpace>

Maps symbols to final predictions via linear layers.

Parameter	Type	Default	Description
<code>nActive</code>	int	<i>required</i>	Number of active output values. For XOR: 1 (single binary output).
<code>nDim</code>	int	1	Dimensionality of each output. Set on <code>TheObjectEncoding</code> ; not passed to the Space constructor.
<code>nVectors</code>	int	= <code>nActive</code>	Codebook size (total vectors in the space). Defaults to <code>nActive</code> for <code>OutputSpace</code> .

Layers: Linear layers with (`bias`, `temp`) support for ergodic mode.

InvertibleLinearLayer

The LDU-factorised invertible linear primitive used internally by `SigmaLayer(invertible=True)` and `PiLayer(invertible=True)`. These parameters are not set via XML directly; they are configured programmatically when constructing the layer.

Parameter	Type	Default	Description
<code>naive</code>	bool	False	False: apply L, D, U sequentially via triangular solves — no W materialisation, backprop through W.
<code>stable</code>	bool	False	Clamps each diagonal entry <code>d_i</code> to magnitude <code>[eps, 1]</code> with sign preserved in <code>_d_effective</code> .
<code>ergodic</code>	bool	False	Enables factor-level noise injection. When <code>True</code> , registers noise buffers <code>noise_raw_L</code> , <code>noise_raw_U</code> .

Class renames and removals.

Old name	New name / status
<code>LULayer</code>	Renamed to <code>InvertibleLinearLayer</code>
<code>InvertibleLinearLayer</code> (SVD-based)	Removed; replaced by LDU factorisation
<code>InvertibleSigmaLayer</code>	Merged into <code>SigmaLayer(invertible=True)</code>
<code>InvertiblePiLayer</code>	Merged into <code>PiLayer(invertible=True)</code>

See `Architecture.md` for the full mathematical treatment of the LDU factorisation and factor-level noise injection.

<architecture><server>

HTTP server settings (used by `serve.py`).

Parameter	Type	Default	Description
<code>host</code>	string	"127.0.0.1"	Server bind address.
<code>port</code>	int	8001	Server port.
<code>timeout</code>	int	120	Request timeout in seconds.

Example: XOR Configuration

```
<model>
  <architecture>
    <reconstruct>symbols</reconstruct>
    <reshape>true</reshape>
    <modelType>lm</modelType>
    <ergodic>>false</ergodic>

    <data>
      <dataset>xor</dataset>
    </data>

    <training>
      <numEpochs>1000</numEpochs>
      <learningRate>0.01</learningRate>
      <autoload>>false</autoload>
      <autosave>>false</autosave>
    </training>
  </architecture>

  <InputSpace>
    <nActive>2</nActive>
    <nDim>1</nDim>
  </InputSpace>

  <PerceptualSpace>
    <nActive>4</nActive>
    <nDim>1</nDim>
    <nVectors>16</nVectors>
    <invertible>>false</invertible>
    <hasAttention>>false</hasAttention>
  </PerceptualSpace>

  <ConceptualSpace>
    <nActive>3</nActive>
    <nDim>1</nDim>
    <nVectors>16</nVectors>
    <invertible>>false</invertible>
  </ConceptualSpace>

  <SymbolicSpace>
    <nActive>3</nActive>
    <passThrough>>true</passThrough>
  </SymbolicSpace>

  <OutputSpace>
    <nActive>1</nActive>
    <nDim>1</nDim>
  </OutputSpace>
</model>
```

In this configuration:

- 2 binary inputs (nActive=2) are lifted, transformed through 4 perceptual and 3 conceptual active

- features
 - PerceptualSpace and ConceptualSpace each have a codebook of 16 vectors (`nVectors=16`), selecting `nActive` via topk
 - 3 symbols are produced: 1 for output prediction, 2 for reconstruction
 - The reverse pass reconstructs the original 2 inputs from all 3 symbols
 - Ergodic mode is off; training uses standard Adam with combined forward+reverse loss
-

Example: Embedding / Chatbot Configuration

```
<model>
  <architecture>
    <reconstruct>symbols</reconstruct>
    <modelType>embedding</modelType>
    <maskedPrediction>ARLM</maskedPrediction>

    <data>
      <dataset>text</dataset>
      <shardDir>data/fineweb</shardDir>
      <numShards>1</numShards>
      <maxDocs>10000</maxDocs>
      <minFrequency>0.00001</minFrequency>
    </data>

    <training>
      <numEpochs>1</numEpochs>
      <batchSize>1</batchSize>
      <learningRate>0.001</learningRate>
      <trainEmbedding>ARLM</trainEmbedding>
      <weightsPath>BasicModel.ckpt</weightsPath>
      <embeddingPath>BasicModel.kv</embeddingPath>
      <autoload>true</autoload>
      <autosave>true</autosave>
      <negSamples>64</negSamples>
    </training>
  </architecture>

  <InputSpace>
    <nActive>128</nActive>
    <nDim>100</nDim>
    <nWhere>2</nWhere>
    <nWhen>2</nWhen>
    <lexer>sentence</lexer>
    <quantized>true</quantized>
  </InputSpace>

  <!-- ... space definitions ... -->
</model>
```

Key points:

- `modelType=embedding` activates the embedding-based input pipeline alongside the neural model
- `trainEmbedding=ARLM` trains only the network layers; the codebook is detached so no gradients flow through it

- The three files partition model behaviour: **XML config** (architecture), **.kv embedding** (codebook), **.ckpt weights** (model layers)
- **weightsPath** stores the neural model checkpoint; **embeddingPath** stores the word vectors
- **minFrequency** gates vocabulary admission: words are buffered until their frequency ratio exceeds this threshold

Installation and Usage

Prerequisites

- **Python 3.12** with a virtual environment at `.venv/`
- **PyTorch** with MPS support (Apple Silicon) or CUDA
- **pandoc** (optional, for PDF generation via `make doc_pdf`)

Setup

```
make venv    # creates .venv with dependencies
```

This installs all Python dependencies (including PyTorch) into a project-local virtual environment. All Makefile targets invoke Python through `.venv/bin/python`, so there is no need to activate the environment manually.

Makefile Targets

Training

Target	Description
<code>make train</code>	Full training (Phase 1 embeddings + Phase 2 model), logged to <code>output/logs/</code>
<code>make train_micro</code>	Small run: 500 docs, 1 random shard, logged
<code>make train_remote</code>	Full training on <code>arbormini.local</code> via SSH
<code>make train_micro_remote</code>	Micro training on <code>arbormini.local</code> via SSH

Models

Target	Description
<code>make run</code>	Run <code>BasicModel.py</code> with the config specified by <code>XML1</code>
<code>make xor</code>	Run with <code>data/XOR_exact.xml</code>
<code>make simple</code>	Run with <code>data/simple.xml</code>
<code>make ergodic</code>	Run with <code>data/ergodic.xml</code>
<code>make tomatoes</code>	Run with <code>data/tomatoes.xml</code>
<code>make mnist</code>	Run with <code>data/mnist.xml</code>
<code>make compare</code>	Compare two models side-by-side using <code>XML1</code> and <code>XML2</code>
<code>make SigmaPi</code>	Run <code>SigmaPi.py</code>
<code>make SymPercept</code>	Run <code>SymPercept.py</code>
<code>make SPNN</code>	Run <code>SPNN.py</code>

Testing and Benchmarks

Target	Description
<code>make test</code>	Run unit tests (forces <code>BASICMODEL_DEVICE=cpu</code>)
<code>make bench</code>	Run training benchmarks (baseline, no env tweaks)

Documentation

Target	Description
<code>make doc_pdf</code>	Generate <code>BasicModel.pdf</code> from doc chapters via pandoc

The PDF is assembled from the following chapters in order: `README.md`, `doc/CognitiveScienceSociety.md`, `doc/Architecture.md`, `doc/BasicModel.md`, `doc/MachineMinds.md`, `doc/Params.md`.

Utilities

Target	Description
<code>make clean</code>	Remove generated files (<code>BasicModel.pdf</code>)
<code>make all</code>	Default target; alias for <code>make xor</code>

Makefile Variables

All variables can be overridden on the command line, e.g. `make run XML1=data/ergodic.xml`.

Variable	Default	Description
<code>MODEL</code>	<code>data/BasicModel.xml</code>	XML config file for training targets
<code>XML1</code>	<code>data/simple.xml</code>	Primary config for <code>make run</code> and <code>make compare</code>
<code>XML2</code>	<code>data/ergodic-only.xml</code>	Secondary config for <code>make compare</code>
<code>TRAIN_HOST</code>	<i>(empty)</i>	Remote training host; set automatically by <code>train_remote</code> targets
<code>TRAIN_USER</code>	<code>arogers</code>	SSH user for remote training
<code>TRAIN_KEY</code>	<code>~/.ssh/id_ed25519_arbormini</code>	SSH private key file
<code>TRAIN_DIR</code>	<code>~/WikiOracle/basicmodel</code>	Working directory on the remote host

train.py Options

`train.py` orchestrates the full training pipeline: Phase 1 (embeddings) followed by Phase 2 (model training).

General

Flag	Default	Description
<code>--model, -m</code>	<code>data/BasicModel.xml</code>	XML config file
<code>--max-docs</code>	<i>(from XML config)</i>	Override <code>maxDocs</code> – limits the number of Wikipedia documents processed
<code>--num-shards</code>	<i>(from XML config)</i>	Override <code>numShards</code> – number of embedding shards to train
<code>--random-shards</code>	<code>off</code>	Pick random shard indices instead of sequential, for variety across runs
<code>--force-embeddings</code>	<code>off</code>	Retrain embeddings even if the output file already exists
<code>--log [FILE]</code>	<code>off</code>	Capture stdout/stderr to a <code>.log</code> file in <code>output/logs/</code> . Omit the filename

Flag	Default	Description
<code>--profile</code>	off	Run Phase 2 under cProfile; writes a <code>.prof</code> file to <code>output/profiles/</code> a

Remote Execution (SSH)

Flag	Default	Description
<code>--host</code>	<i>(none)</i>	Remote host (e.g. <code>arbormini.local</code>). When set, training runs remotely.
<code>--user</code>	<code>arogers</code>	SSH user
<code>--key-file</code>	<code>~/.ssh/id_ed25519_arbormini</code>	SSH private key
<code>--remote-dir</code>	<code>~/WikiOracle/basicmodel</code>	Working directory on the remote machine

Environment Variables

Variable	Description
<code>BASIC_MAX_DOCS</code>	Override <code>maxDocs</code> in <code>BasicModel.py</code> (set automatically by <code>train.py</code> when <code>--max-</code>
<code>BASIC_NUM_SHARDS</code>	Override <code>numShards</code> in <code>BasicModel.py</code> (set automatically by <code>train.py</code> when <code>--nu</code>
<code>BASICMODEL_DEVICE</code>	Force a specific device, e.g. <code>cpu</code> . Used by <code>make test</code> to avoid GPU-dependent fail
<code>PYTORCH_MPS_HIGH_WATERMARK_RATIO</code>	MPS memory allocation limit. Set to 0.0 by <code>train.py</code> to disable the high-waterm
<code>PYTHONUNBUFFERED</code>	Disable Python output buffering. Set to 1 by <code>train.py</code> so log output streams in re

Remote Training (ArborMini)

The `train_remote` and `train_micro_remote` targets automate the full remote-training workflow. Under the hood they set `TRAIN_HOST=arbormini.local` and delegate to `train.py --host`.

Workflow

1. **Rsync** syncs the local project tree to `arbormini.local`:

```
rsync -av --progress \
  -e "ssh -i ~/.ssh/id_ed25519_arbormini" \
  --exclude .venv \
  --exclude __pycache__ \
  --exclude .DS_Store \
  --exclude output/ \
  ./ arogers@arbormini.local:~/WikiOracle/basicmodel/
```

The exclusions keep the transfer fast: `.venv/` (the remote has its own), `__pycache__/` (bytecode), `.DS_Store` (macOS metadata), and `output/` (embeddings, logs, profiles).

2. **SSH** connects to ArborMini and runs `train.py` with the same flags that were passed locally (minus `--host`):

```
ssh -i ~/.ssh/id_ed25519_arbormini -t arogers@arbormini.local \
  "cd ~/WikiOracle/basicmodel && PYTHONUNBUFFERED=1 PYTHONPATH=bin .venv/bin/python bin/train.py --"
```

3. **Output stays remote.** Profile data (`.prof` files) and training logs remain on ArborMini under `~/WikiOracle/basicmodel/output/`.

SSH Key Setup

Remote targets require passwordless SSH access. Generate and copy a key if you have not already:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_arbormini  
ssh-copy-id -i ~/.ssh/id_ed25519_arbormini arogers@arbormini.local
```

Quick Start

```
# Fast sanity check: 500 docs, 1 shard, on ArborMini  
make train_micro_remote
```

```
# Full training run on ArborMini  
make train_remote
```